



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Analyse de la consommation énergétique du DNS

Rapport de stage ST50 - P2026

LEROY Fernand

FISE-INFO
Data Science et IA

Télécom SudParis

19 Place Marguerite Perey
91120 Palaiseau
www.telecom-sudparis.eu

Tuteur en entreprise
MAROT Michel

Suiveur UTBM
BAALA CANALDA Oumaya

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Remerciements

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Table des matières

Introduction	4
1 Présentation de Télécom SudParis	5
1.1 Présentation de l'organisme d'accueil	5
1.1.1 Télécom SudParis dans son environnement international, européen et français	5
1.1.2 Télécom SudParis au sein de l'Institut Mines-Télécom et de l'Institut Polytechnique de Paris	5
1.1.3 Le département Réseaux et Services de Télécommunications (RST)	6
1.2 Environnement de travail	6
2 Contexte théorique	8
2.1 Fonctionnement du DNS	8
2.1.1 Serveurs autoritatifs et résolveurs	8
2.1.2 Résolution récursive	9
2.1.3 Le mécanisme de cache	9
2.2 Protocoles utilisés	9
2.2.1 DNS sur UDP	9
2.2.2 DNS over TLS (DoT)	10
2.2.3 DNS over HTTPS (DoH)	10
2.3 Présentation de Bind9	10
3 Travail réalisé	11
3.1 Description du projet DECODES	11
3.1.1 Organisation du travail	11
3.2 État de l'art	12
3.3 Déroulement du travail	12
3.3.1 Conception de l'environnement expérimental	12
3.3.2 Campagnes de mesures	18
3.3.3 Analyse des résultats	19
3.3.4 Modélisation	34
Conclusion	39
Bibliographie	39
Annexes	41
Détail des métriques collectées	41
3.4 Types de requêtes DNS	42

Introduction

Contexte :

Les DNS sont utilisés quotidiennement. **Problème :**

Nous devons trouver une méthode pour mesurer et estimer la consommation énergétique du DNS, incluant tous les acteurs différents présents dans le processus de résolution d'un nom de domaine. Mesurer un système distribué comme le DNS n'est pas simple et implique l'estimation de la consommation énergétique de tous les acteurs du système. Pour simplifier le problème, nous nous sommes d'abord concentrés sur l'estimation de la consommation énergétique du premier acteur : le résolveur.

Travaux connexes : Ce document s'appuie sur les travaux d'anciens stagiaires ayant mesuré la même configuration : *Analyse de la consommation énergétique des résolveurs DNS* John Doe. This is a book. Sous la direction d'Editor. Publisher, 2023.

Défis ou lacunes : Les études précédentes sur la consommation énergétique des résolveurs DNS manquent d'interprétabilité. Celle-ci est importante pour modéliser cette consommation et estimer la consommation énergétique de différents types de résolveurs à différentes échelles. De plus, contrairement à d'autres études, la méthode utilisée pour mesurer la consommation énergétique du cache du résolveur utilise le même ensemble de domaines que celui utilisé pour mesurer le résolveur sans cache. **Cela permet une estimation de la consommation énergétique plus proche des cas d'usage réels.** Pour l'estimation de la consommation énergétique de la résolution sans utilisation du cache, la configuration est modifiée pour empêcher la mise en cache des domaines qui pourraient réapparaître et fausser les résultats.

Idée principale : Comparaison de deux approches pour modéliser la consommation énergétique de Bind9, une implémentation de résolveur DNS. La première consiste à analyser son fonctionnement pour créer un modèle de file d'attente qui nous aidera à estimer la consommation énergétique. La deuxième approche est agnostique : la complexité algorithmique du logiciel est analysée temporellement et spatialement, de manière statique et pendant l'exécution. // Il faudrait une petite comparaison entre les deux approches pour expliquer leurs différences et limitations.

Contributions :

- Méthode d'estimation de la consommation énergétique des logiciels/DNS
- Étude de la consommation via un modèle de file d'attente
- Étude de la consommation via une analyse du code d'une implémentation DNS (Bind9)

Organisation : du document / rapport

Chapitre 1

Présentation de Télécom SudParis

1.1 Présentation de l'organisme d'accueil

1.1.1 Télécom SudParis dans son environnement international, européen et français

Télécom SudParis est une grande école d'ingénieurs française spécialisée dans les technologies du numérique, les télécommunications, les réseaux, l'informatique et la cybersécurité. Depuis 2019, elle est membre de l'Institut Polytechnique de Paris (IP Paris), qui rassemble plusieurs grandes écoles françaises de premier plan telles que l'École Polytechnique, Télécom Paris, l'ENSTA Paris et l'ENSAE Paris. Cette intégration lui permet de bénéficier d'une forte visibilité académique et scientifique à l'échelle internationale.

Au niveau européen, Télécom SudParis participe à de nombreux projets de recherche et collabore avec des universités, laboratoires et entreprises du secteur numérique. Ses activités de recherche couvrent notamment les réseaux de communication, la cybersécurité, l'intelligence artificielle et les systèmes distribués.

Au niveau national, l'école appartient également à l'Institut Mines-Télécom (IMT), premier groupe public français d'écoles d'ingénieurs et de management dédié à la transformation numérique, industrielle et énergétique. Télécom SudParis contribue ainsi à la formation d'ingénieurs et de chercheurs capables de répondre aux enjeux technologiques actuels et futurs.

1.1.2 Télécom SudParis au sein de l'Institut Mines-Télécom et de l'Institut Polytechnique de Paris

Fondée en 1979 sous le nom d'Institut National des Télécommunications (INT), Télécom SudParis est aujourd'hui l'une des écoles de l'Institut Mines-Télécom tout en étant membre de l'Institut Polytechnique de Paris. Elle a pour mission de former des ingénieurs, chercheurs et experts du numérique grâce à un enseignement de haut niveau associé à une activité de recherche reconnue.

L'école est organisée autour de plusieurs départements d'enseignement et de recherche couvrant différents domaines du numérique, tels que les réseaux, l'informatique, les systèmes embarqués ou encore la cybersécurité. Elle entretient de nombreux partenariats avec des acteurs industriels et académiques, en France comme à l'étranger.

Durant mon stage, j'ai été accueilli sur le campus de Palaiseau, situé au sein du cluster scientifique de l'Institut Polytechnique de Paris. Bien que Télécom SudParis possède historiquement un campus principal à Évry-Courcouronnes, une partie de ses activités d'ensei-

gnement et de recherche est aujourd'hui implantée sur le campus de Palaiseau, notamment dans les locaux partagés avec Télécom Paris.

1.1.3 Le département Réseaux et Services de Télécommunications (RST)

Mon stage s'est déroulé au sein du département **Réseaux et Services de Télécommunications (RST)** de Télécom SudParis. Ce département constitue le pôle d'expertise de l'école dans le domaine des réseaux informatiques et des télécommunications. Il est implanté à la fois sur les campus d'Évry-Courcouronnes et de Palaiseau.

Le département RST intervient aussi bien dans l'enseignement que dans la recherche. Il assure la coordination des formations liées aux réseaux et propose notamment des spécialisations en sécurité des systèmes et réseaux ainsi qu'en architecture et intelligence pour les réseaux.

Ses activités de recherche portent sur de nombreux domaines tels que les protocoles de communication, la cybersécurité, la qualité de service, la virtualisation des réseaux, l'optimisation des infrastructures numériques ou encore les réseaux du futur. Les enseignants-chercheurs du département sont notamment associés au laboratoire SAMOVAR et collaborent avec de nombreux partenaires académiques et industriels, à l'échelle nationale et internationale.

C'est dans cet environnement de recherche et d'innovation, spécialisé dans les réseaux et les télécommunications, que s'est déroulé mon stage.

1.2 Environnement de travail

Ce stage s'est déroulé dans un environnement de recherche académique, sensiblement différent du cadre industriel ou entrepreneurial. Réalisé au sein du département Réseaux et Services de Télécommunications (RST) de Télécom SudParis, il s'inscrivait dans une démarche de recherche publique dont l'objectif principal est la production de connaissances et l'étude approfondie de problématiques scientifiques.

Contrairement à un contexte industriel, où les contraintes de temps et de rentabilité occupent souvent une place centrale, la recherche académique privilégie une approche davantage fondée sur l'analyse, l'expérimentation et la validation rigoureuse des résultats. Cette méthodologie favorise l'exploration de différentes pistes de travail et laisse une plus grande place à la réflexion scientifique.

Télécom SudParis est implantée sur deux sites principaux : Évry-Courcouronnes et Palaiseau. Mon activité s'est déroulée majoritairement sur le campus de Palaiseau, au sein des locaux partagés avec Télécom Paris. Ce campus récent offre un cadre de travail moderne et agréable, caractérisé par des espaces lumineux et des infrastructures adaptées aux activités d'enseignement et de recherche.

J'ai également été amené à me rendre ponctuellement sur le site d'Évry-Courcouronnes, où étaient hébergés les serveurs utilisés pour les expérimentations. Ces déplacements ont notamment permis la configuration initiale des équipements ainsi que des échanges avec les membres de l'équipe technique.

Enfin, je disposais d'espaces de travail sur les deux sites, partagés avec d'autres stagiaires et doctorants. Cet environnement calme et collaboratif a favorisé la concentration et le bon déroulement des activités de recherche et de développement liées au projet.

Ce cadre de travail m'a permis de découvrir le fonctionnement d'un environnement de recherche académique et de développer une plus grande autonomie dans la conduite d'un

projet scientifique. La flexibilité de l'organisation, associée à un encadrement régulier, m'a offert l'opportunité d'explorer différentes approches méthodologiques tout en conservant une démarche rigoureuse. Cette expérience a constitué une introduction concrète au travail de recherche et a fourni le contexte nécessaire à la réalisation des travaux présentés dans les chapitres suivants.

Chapitre 2

Contexte théorique

Afin de comprendre les travaux réalisés au cours de ce stage, il est nécessaire de présenter les principaux concepts liés au système de noms de domaine (DNS), aux protocoles utilisés pour le transport des requêtes DNS ainsi qu'au logiciel Bind9, qui constitue la plateforme d'expérimentation principale de cette étude.

2.1 Fonctionnement du DNS

Le *Domain Name System* (DNS) est un service essentiel d'Internet dont le rôle principal est de traduire les noms de domaine compréhensibles par les utilisateurs, tels que `www.example.com`, en adresses IP utilisables par les équipements réseau.

Le DNS repose sur une architecture distribuée et hiérarchique. Chaque niveau de cette hiérarchie est responsable d'une partie de l'espace des noms de domaine, ce qui permet au système de rester scalable malgré la taille d'Internet.

Complete DNS Lookup and Webpage Query

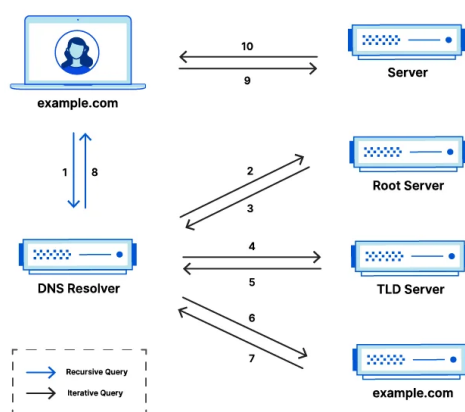


Figure 2.1 – Exemple simplifié d'une résolution DNS récursive.

2.1.1 Serveurs autoritatifs et résolveurs

L'infrastructure DNS est principalement composée de deux types de serveurs.

Les **serveurs autoritatifs** stockent les informations officielles relatives à un ou plusieurs domaines. Ils constituent la source de vérité concernant les enregistrements DNS qu'ils hébergent.

Les **résolveurs DNS** ont pour rôle de répondre aux requêtes des clients. Lorsqu'une information n'est pas disponible localement, ils interrogent successivement les différents serveurs de la hiérarchie DNS jusqu'à obtenir une réponse qu'ils retransmettent au client.

Dans le cadre de ce stage, les travaux portent principalement sur les résolveurs DNS, car ils représentent le point d'entrée des requêtes générées par les utilisateurs et intègrent des mécanismes de cache susceptibles d'influencer significativement leur consommation énergétique.

2.1.2 Résolution récursive

La résolution récursive correspond au processus par lequel un résolveur recherche une information DNS au nom du client.

Lorsqu'un utilisateur effectue une requête, celle-ci est transmise à un résolveur. Si la réponse n'est pas déjà disponible dans son cache, le résolveur contacte successivement les serveurs racine, puis les serveurs responsables du domaine de premier niveau (*Top-Level Domain*) concerné, avant d'interroger le serveur autoritatif du domaine recherché.

Une fois la réponse obtenue, elle est renvoyée au client et peut être conservée temporairement dans le cache du résolveur afin d'accélérer les futures requêtes similaires.

2.1.3 Le mécanisme de cache

Le cache constitue un élément fondamental du fonctionnement des résolveurs DNS. Lorsqu'une réponse est obtenue auprès d'un serveur autoritatif, elle est conservée localement pendant une durée définie par son *Time To Live* (TTL).

Si une requête identique est reçue avant l'expiration de cette durée, le résolveur peut répondre directement à partir du cache sans effectuer de nouvelles communications réseau.

Ce mécanisme permet de réduire la latence des requêtes, de diminuer la charge des serveurs autoritatifs et de limiter le trafic réseau généré par les résolutions DNS. Dans le cadre de ce stage, l'étude de l'impact énergétique du cache constitue un élément central des expérimentations réalisées.

2.2 Protocoles utilisés

Traditionnellement, les échanges DNS utilisent le protocole UDP. Cependant, l'apparition de nouvelles exigences en matière de confidentialité et de sécurité a conduit au développement de protocoles chiffrés tels que DNS over TLS (DoT) et DNS over HTTPS (DoH).

2.2.1 DNS sur UDP

Le protocole UDP (*User Datagram Protocol*) est historiquement le mode de transport le plus utilisé pour les requêtes DNS.

Son principal avantage réside dans sa simplicité : aucune connexion préalable n'est nécessaire entre le client et le serveur, ce qui réduit la surcharge protocolaire et permet d'obtenir de très faibles temps de réponse.

Cependant, les données sont transmises en clair sur le réseau, ce qui les rend observables par des intermédiaires.

2.2.2 DNS over TLS (DoT)

Le protocole DNS over TLS encapsule les échanges DNS dans une connexion TLS.

Cette approche garantit la confidentialité et l'intégrité des données échangées entre le client et le résolveur. En contrepartie, l'établissement et le maintien de la connexion TLS introduisent un coût supplémentaire en termes de calcul et de consommation de ressources.

2.2.3 DNS over HTTPS (DoH)

Le protocole DNS over HTTPS transporte les requêtes DNS au travers du protocole HTTPS.

En plus du chiffrement apporté par TLS, cette approche permet aux requêtes DNS de se fondre dans le trafic Web classique. Cette caractéristique améliore la confidentialité des utilisateurs mais ajoute également une couche protocolaire supplémentaire susceptible d'avoir un impact sur les performances et la consommation énergétique des serveurs.

L'une des motivations de ce stage est précisément d'évaluer l'influence de ces différents protocoles sur la consommation des infrastructures DNS.

2.3 Présentation de Bind9

Les expérimentations réalisées durant ce stage reposent principalement sur Bind9 (*Berkeley Internet Name Domain*), l'un des logiciels DNS les plus utilisés au monde.

Développé par l'ISC (*Internet Systems Consortium*), Bind9 est une implémentation open source permettant de déployer aussi bien des serveurs DNS autoritatifs que des résolveurs récursifs. Sa large adoption dans les environnements académiques, industriels et institutionnels en fait une référence pertinente pour l'étude du comportement énergétique des infrastructures DNS.

Bind9 offre de nombreuses fonctionnalités avancées, notamment la gestion du cache, le support des protocoles DNS modernes tels que DoT et DoH, ainsi que des outils détaillés de supervision et de collecte de statistiques. Ces caractéristiques en font une plateforme particulièrement adaptée à la réalisation de mesures expérimentales et à la construction de modèles de consommation énergétique.

L'ensemble des mesures et analyses présentées dans les chapitres suivants a ainsi été réalisé à partir de différentes configurations de Bind9 exécutées sur les serveurs du projet DECODES.

Chapitre 3

Travail réalisé

Introduction

3.1 Description du projet DECODES

Décrire le sujet défini à mon arrivée et les objectifs généraux du projet. Mon rôle dans le projet. (organigramme des gens du projet et de leurs rôles) Décrire le sujet réel (analyse de la complexité et modélisation)

3.1.1 Organisation du travail

Le stage s'inscrivait dans le cadre du projet DECODES (*Datacenter Energy Consumption Optimisation by DNS EvaluationS*), mené en collaboration avec plusieurs partenaires académiques et industriels, notamment l'AFNIC, IXFrance, l'ENPC et Télécom SudParis.

L'organisation du travail reposait sur une large autonomie dans la conduite des activités de recherche. Les objectifs généraux étaient définis par les encadrants, mais le choix des méthodologies, des outils et des protocoles expérimentaux relevait en grande partie de mon initiative. Cette liberté m'a permis d'explorer différentes approches et d'adapter progressivement mes travaux en fonction des résultats obtenus.

Le suivi du stage était assuré au travers de réunions régulières avec mes encadrants, généralement organisées de manière hebdomadaire. Ces échanges permettaient de présenter l'avancement des travaux, de discuter des difficultés rencontrées et d'orienter les prochaines étapes de la recherche.

J'ai également participé à plusieurs réunions plénières du projet DECODES réunissant l'ensemble des partenaires. Ces rencontres ont constitué une opportunité de présenter les résultats intermédiaires obtenus, de recueillir des retours extérieurs et de mieux comprendre les enjeux globaux du projet.

Les travaux réalisés au cours du stage se sont articulés autour de quatre grandes phases :

1. Étude bibliographique

Une première phase a consisté à acquérir une compréhension approfondie du sujet à travers l'étude de la littérature scientifique relative au DNS, aux infrastructures réseau et aux méthodes de mesure de la consommation énergétique.

2. Conception de l'environnement expérimental

Cette étape a porté sur la mise en place des outils nécessaires aux expérimentations : développement de scripts de mesure, configuration des serveurs de test et validation des protocoles de collecte de données.

3. Campagnes de mesures et analyse des résultats

Les expérimentations ont ensuite été conduites de manière itérative afin d'évaluer différentes configurations et d'améliorer progressivement la qualité des mesures obtenues.

4. Modélisation et valorisation des travaux

Enfin, les résultats collectés ont été exploités afin de construire des modèles d'analyse et de documenter les conclusions du stage au travers de rapports, présentations et travaux de rédaction.

3.2 État de l'art

3.3 Déroulement du travail

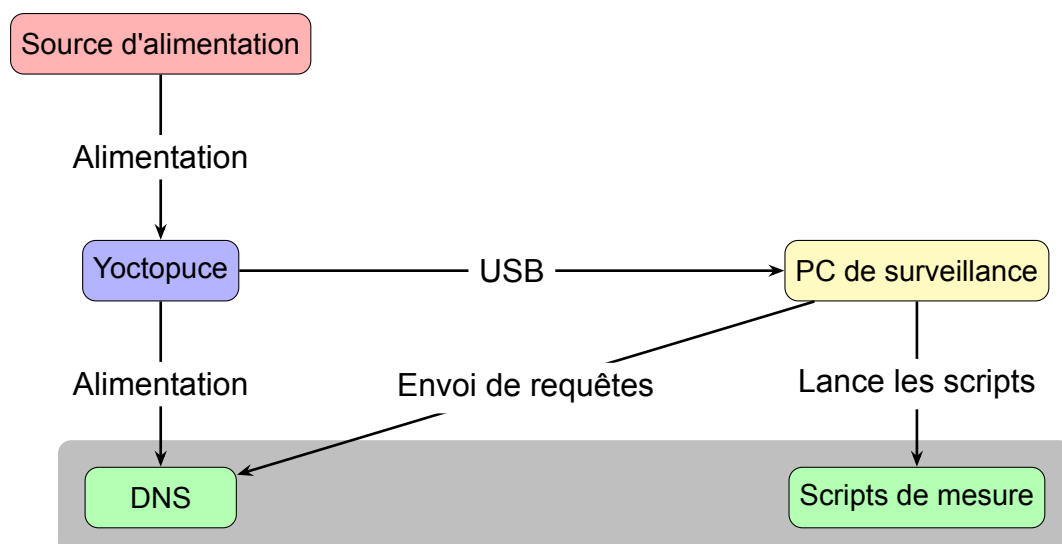
3.3.1 Conception de l'environnement expérimental

La première partie de cette recherche consiste à mesurer de manière efficace et détaillée la consommation d'un résolveur DNS. Pour ce faire, j'ai choisi une approche boîte noire, en mesurant le plus grand nombre de métriques possibles, puis en déduisant celles qui sont les plus pertinentes pour estimer la consommation énergétique.

Configuration des différentes machines

La configuration expérimentale repose sur deux machines : un client et un serveur DNS. Le client joue également le rôle d'orchestrateur et est chargé de déclencher ainsi que de coordonner l'exécution des mesures sur l'ensemble du système. La consommation énergétique externe est mesurée à l'aide d'un dispositif Yoctopuce (wattmètre) et est supervisée directement depuis la machine cliente.

Le serveur DNS est basé sur une implémentation de Bind9 et exécute, en parallèle du traitement des requêtes, des scripts dédiés à la collecte de différentes métriques de performance. Ces scripts introduisent une charge supplémentaire non négligeable, susceptible d'influencer les mesures de consommation énergétique, et doivent donc être pris en compte dans l'interprétation des résultats. Dans cette architecture, le client assure ainsi simultanément les fonctions de générateur de charge et d'orchestrateur des campagnes de mesures.



Outils de mesure utilisés

Un grand nombre de métriques sont collectées pour aider à estimer et modéliser la consommation énergétique du résolveur DNS. Ces métriques peuvent se classer dans 4 catégories : les mesures de critères systèmes, les mesures de consommation énergétiques externes, les logs réseau et les mesures spécifiques au logiciel (Bind9). Pour collecter ces métriques les outils utilisés selon la catégories sont :

- **Mesures de critères systèmes** : `turbostat` (CPU), mesures système données par le kernel (I/O, NIC,), `perf` (RAM)
- **Mesures de consommation énergétique externes** : `Yoctopuce`
- **Logs réseau** : `tcpdump`
- **Mesures spécifiques à Bind9** : `rndc` (Informations sur le cache de Bind9), `uftrace` (Rapports des fonctions exécutées par Bind9)

Les outils de mesures de critères systèmes ont été sélectionnés à l'aide d'un rapport de stage écrit par une étudiante ayant fait une étude comparative de différents outils de mesure de consommation énergétique de serveurs John Doe. This is a book. Sous la direction d'Editor. Publisher, 2023. Le détail des métriques collectées par ces outils est présenté dans les annexes.

Concernant les outils de mesure spécifiques à Bind9, `rndc` est un outil de contrôle de Bind9 qui permet d'obtenir des informations sur le cache, les statistiques de requêtes, etc. `uftrace` est un outil de traçage de fonctions qui permet d'obtenir des rapports détaillés sur les fonctions exécutées par des logiciels comme Bind9, ce qui est particulièrement utile pour l'analyse de la complexité algorithmique.

Scripts de mesure développés

Afin de mesurer les métriques nécessaires à l'évaluation de la consommation énergétique du résolveur DNS et à la modélisation de celle-ci, plusieurs scripts de collecte ont été développés. Ces scripts permettent d'extraire des indicateurs système, réseau et énergétiques de manière synchronisée sur l'ensemble de la chaîne de mesure.

- **Scripts de mesure des indicateurs système** :
 - `measure_cpu.sh` : utilise l'outil `turbostat` afin de collecter des informations relatives à l'utilisation du processeur, notamment la fréquence, les états de sommeil (C-states) ainsi que les compteurs de performance associés à l'activité CPU.
 - `measure_io.sh` : exploite les fichiers `/sys/block/sda/stat`, qui contiennent des compteurs cumulés depuis le démarrage du système pour les opérations d'E/S disque (lecture et écriture en kilo-octets). Les valeurs sont différenciées temporellement afin d'obtenir les volumes d'I/O sur chaque intervalle de mesure.
 - `measure_nic.sh` : repose sur les compteurs du noyau Linux situés dans `/sys/class/net/eno1` notamment `rx_bytes` et `tx_bytes`, représentant respectivement le volume de données reçues et transmises par l'interface réseau. Comme pour les I/O disque, une différenciation temporelle permet d'obtenir les débits sur les intervalles considérés.
 - `measure_ram.sh` : collecte des métriques liées à l'utilisation mémoire via les opérations sur le Last Level Cache (LLC), notamment les événements `LLC-loads` et `LLC-stores`. Ces indicateurs sont utilisés comme estimateurs indirects de l'activité mémoire et des accès à la RAM.
- **Scripts de collecte de logs réseau** :
 - `dns_activity_logger.sh` : capture le trafic réseau à l'aide de `tcpdump` afin d'analyser les requêtes DNS reçues par le résolveur ainsi que les réponses émises. Ces données permettent de reconstruire le flux de requêtes et d'évaluer la charge effective.

- `bind_query_capture.sh` : exploite les journaux de Bind9 afin d'identifier les requêtes effectivement traitées par le résolveur et d'obtenir une vision logique des résolutions effectuées (indépendamment du niveau réseau brut).
- **Script de collecte des mesures externes :**
 - `yoctowatt_log_fast.py` : utilise un dispositif Yoctopuce (wattmètre) afin de mesurer en continu la consommation énergétique du serveur exécutant le résolveur DNS. Les mesures sont synchronisées avec les autres sources de données afin de permettre une corrélation temporelle précise.
- **Mesures liées à Bind9 :**
 - `monitor_cache.sh` : interroge Bind9 via `rndc` afin d'extraire les métriques internes du cache DNS (taux de hit/miss, taille du cache, mémoire utilisée, etc.).
 - `record_ufrtrace.sh` : utilise `uftrace` pour obtenir des rapports détaillés sur les fonctions exécutées par Bind9 et leur nombre d'exécutions, ce qui permet d'analyser la complexité algorithmique dynamique et d'identifier les points chauds en termes de consommation de ressources.

L'ensemble de ces scripts constitue le squelette qui va être utilisé pour la collecte de données nécessaire à la modélisation de la consommation énergétique du résolveur DNS. Ils permettent d'obtenir une vue multi-niveaux du système (CPU, mémoire, I/O, réseau et cache DNS), ainsi qu'une mesure directe de la consommation électrique. Ces données serviront de base à la construction des modèles, à l'estimation des paramètres et à l'identification des facteurs influençant la consommation énergétique.

Sélection de l'ensemble des noms de domaines utilisés

Pour faire les mesures de consommation du DNS, il faut envoyer des requêtes de résolution de noms de domaines. Un ensemble de 17 millions de noms de domaine est accessible sur GitHub avec leur popularité **DomainNameDataset**.

Avec autant de noms de domaines, on peut se demander si avoir un ensemble de 17 millions de noms de domaines est nécessaire pour faire les mesures, ou si un ensemble plus petit pourrait être suffisant. La taille de l'ensemble de noms de domaines a-t-elle une influence sur les mesures de consommation énergétique du DNS ?

Une autre question se pose, cela n'a pas encore été abordé mais pour faire les mesures de consommation du DNS sans utilisation du cache, comment faire ? Les précédents travaux ont utilisé un seul nom de domaine pour simuler l'utilisation du cache et un grand nombre de noms de domaines (360 000) pour simuler l'utilisation sans cache. Cela permet en effet de voir l'influence du cache sur la consommation énergétique du DNS, mais cela ne correspond pas à un cas d'usage réel, de plus les conditions de mesure ne sont pas les mêmes pour les deux configurations, ce qui peut fausser les résultats.

Premièrement, pour voir l'influence de la taille de l'ensemble de noms de domaines sur la consommation énergétique du DNS, il est intéressant de regarder si le cache peut être saturé, et si c'est possible à combien de noms de domaines cela correspond-t-il en moyenne ? Pour savoir ça, j'ai fait des mesures de différentes métriques associées au cache de Bind9 pour différents ensembles de noms de domaines entre 100 000 et 14 millions de noms de domaines. En premier lieu avec des ensembles de noms de domaines entre 100 000 et 1 million de noms de domaines avec un interval de 100 000 noms de domaines entre chaque mesure, puis avec des ensembles de noms de domaines entre 1 et 14 millions de noms de domaines avec un interval de 1 million de noms de domaines entre chaque mesure.

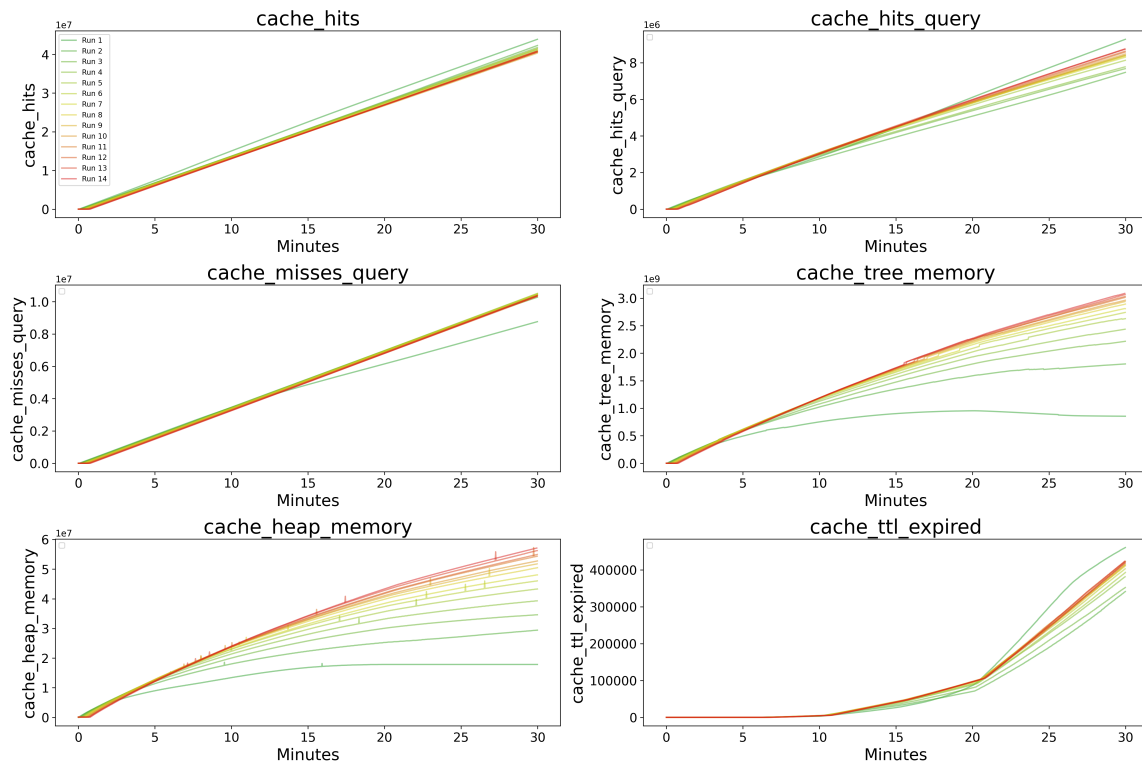


Figure 3.1 – Évolution des valeurs associées au cache de bind pour différents ensemble de noms de domaines entre 100 000 et 1 millions de noms de domaines

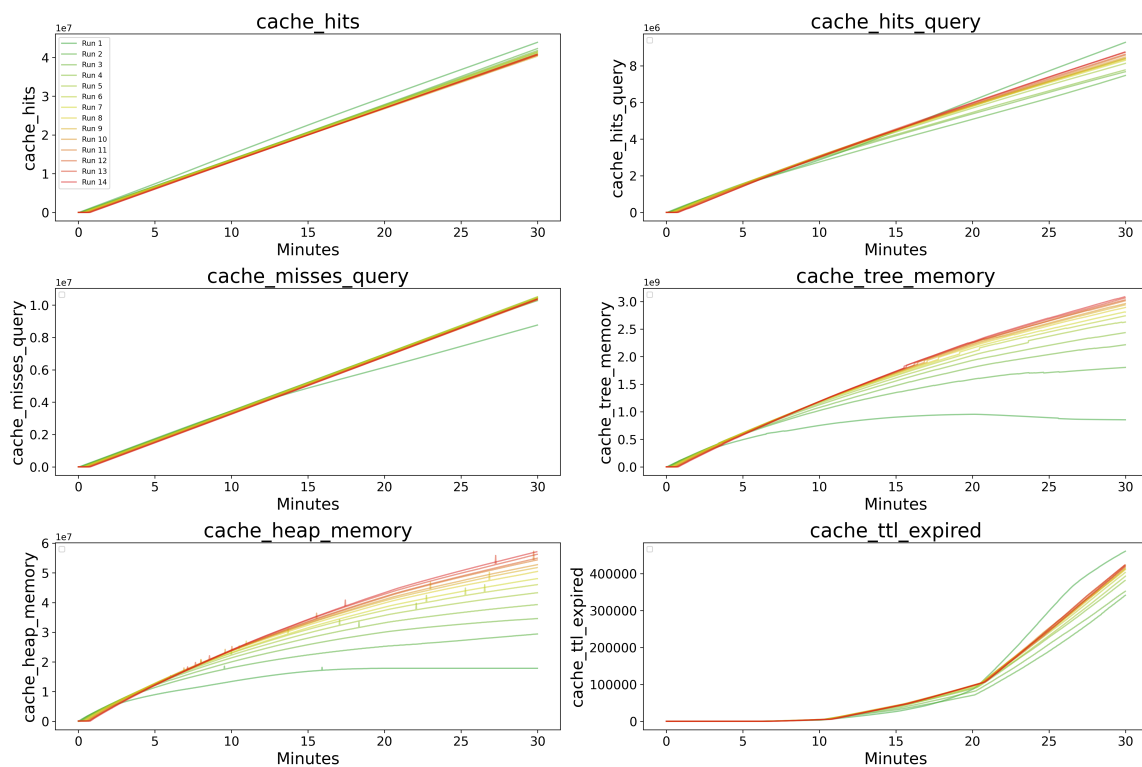


Figure 3.2 – Évolution des valeurs associées au cache de bind pour différents ensemble de noms de domaines entre 1 et 14 millions de noms de domaines

Les mesures ne vont pas au delà de 14 millions de noms de domaines car la RAM du serveur client n'était pas suffisante pour faire les mesures avec des ensembles de noms de

domaines plus grands et cela ne changeait pas significativement les résultats.

Les 6 métriques mesurées sont :

- **cache_hits** : nombre total de réponses servies depuis le cache
- **cache_hits_query** : nombre de requêtes résolues directement depuis le cache
- **cache_misses_query** : nombre de requêtes nécessitant une résolution externe
- **cache_tree_memory** : mémoire utilisée par la structure arborescente du cache
- **cache_heap_memory** : mémoire utilisée par le tas du cache
- **cache_ttl_expired** : nombre d'entrées supprimées après expiration du TTL

La métrique **cache_mem_exhaust** n'est pas représentée sur les figures car sa valeur reste nulle durant l'intégralité des mesures. Ce résultat indique qu'aucune saturation du cache n'est observée, y compris pour l'ensemble de données le plus important, composé de 14 millions de noms de domaine.

L'évolution des métriques **cache_tree_memory** et **cache_heap_memory** met en évidence le remplissage progressif du cache au cours du temps. Plus la taille de l'ensemble de données est importante, plus la durée nécessaire à la mise en cache de l'ensemble des entrées est élevée.

Les expérimentations ont été réalisées pendant 30 minutes en utilisant le protocole UDP avec une charge constante de 12,000 requêtes par seconde. Une diminution progressive du nombre d'entrées présentes dans le cache est observée à partir d'environ 15 minutes, en raison de l'expiration des enregistrements associée au TTL. Ce phénomène devient particulièrement marqué après 20 minutes. Son ampleur augmente avec la taille de l'ensemble de données, ce qui s'explique par le nombre plus important d'entrées susceptibles d'atteindre leur durée de vie maximale.

Par ailleurs, le nombre de **cache_hits_query** augmente continuellement au cours des mesures pour l'ensemble des jeux de données. Cette augmentation est toutefois plus rapide pour les ensembles de petite taille, le cache atteignant plus rapidement un état stable dans lequel une proportion importante des requêtes peut être satisfaite localement. À l'inverse, le nombre de **cache_misses_query** diminue progressivement au fil du temps, avec une décroissance plus prononcée pour les ensembles de petite taille.

Ces tendances apparaissent plus nettement sur les courbes de la figure 3.1 que sur celles de la figure 3.2. Étant donné que la durée des mesures est limitée à 30 minutes, les ensembles de grande taille ne permettent pas d'atteindre un niveau de remplissage du cache comparable à celui observé pour les ensembles plus réduits. Les variations des métriques sont donc plus progressives et les courbes présentent une pente plus faible durant la phase initiale de l'expérience.

Il convient de noter que, même lorsque l'ensemble des noms de domaine a été chargé dans le cache, des **cache_misses_query** continuent d'être observés. Ce comportement s'explique principalement par les requêtes supplémentaires générées lors des mécanismes de validation DNSSEC ainsi que par l'expiration régulière des entrées dont la durée de vie (TTL) est atteinte. Ces événements nécessitent la réalisation de nouvelles résolutions externes et contribuent ainsi au maintien d'un nombre non nul de défauts de cache.

L'analyse des courbes permet également d'estimer le temps nécessaire au remplissage du cache. Pour un ensemble de 100,000 noms de domaine, le cache atteint un état proche de la saturation après environ 1 minute et 30 secondes. Cette durée correspond au temps requis pour que la majorité des entrées de l'ensemble de données soient résolues puis stockées localement dans le cache du résolveur.

Pour répondre à la question de l'influence de la taille de l'ensemble de noms de domaine sur la consommation énergétique du DNS, des mesures ont été réalisées pour différents ensembles de noms de domaine entre 100,000 et 1 millions :

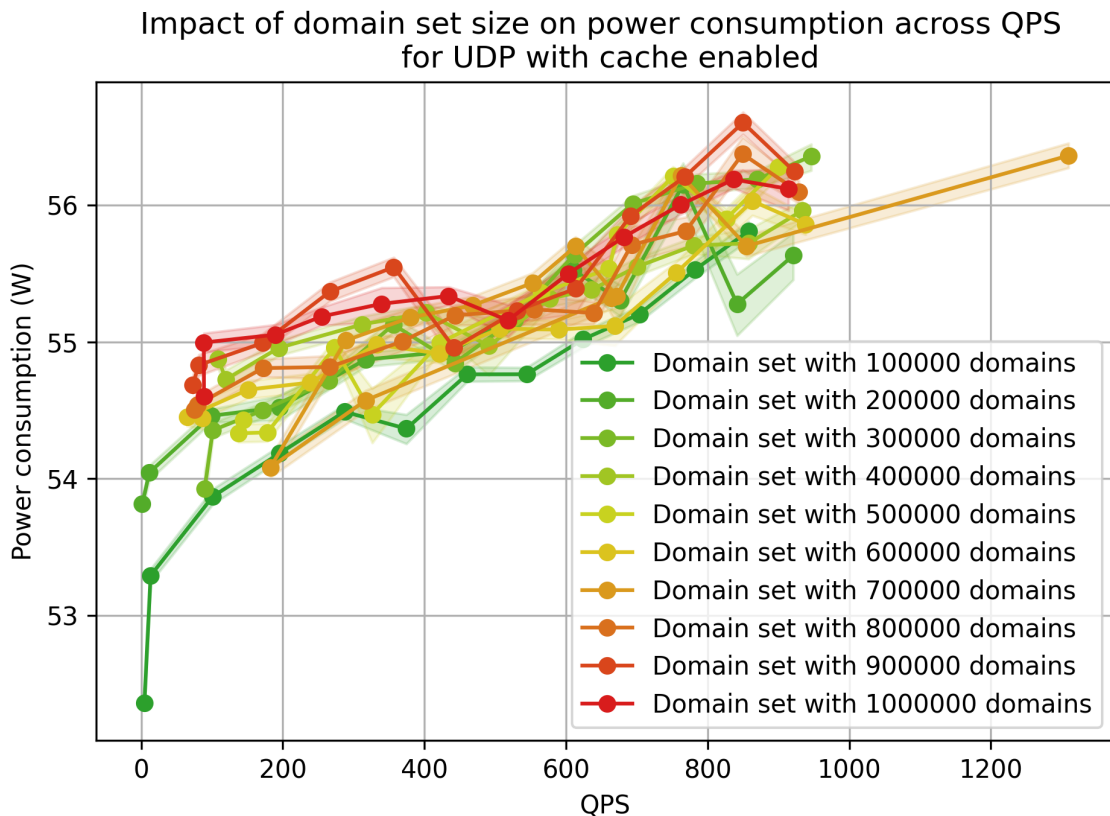


Figure 3.3 – Consommation énergétique moyenne pour différents ensembles de noms de domaine entre 100,000 et 1 million de noms de domaine

Ces mesures ont été perturbées par des ip externes envoyant des requêtes au serveur DNS durant les mesures, ce qui a faussé les résultats. Cependant, on peut observer une petite différence de consommation énergétique entre les différentes tailles d'ensemble de noms de domaine, mais cette différence n'est pas significative.

Gestion des mesures avec et sans cache

Le protocole retenu pour la réalisation des mesures consiste, dans un premier temps, à préremplir le cache pendant une durée déterminée, suffisante pour permettre le chargement de l'ensemble des entrées associées au jeu de données de noms de domaine utilisé. Cette phase vise à garantir que les mesures effectuées avec cache reflètent le comportement du résolveur dans un état où le cache est déjà établi. Le nombre de noms de domaines choisi est 100,000 pour permettre un remplissage rapide du cache et ainsi limiter la durée totale des campagnes de mesures.

Pour les mesures réalisées sans utilisation du cache, une option de configuration de BIND a été employée afin de désactiver artificiellement le mécanisme de mise en cache. Il s'agit du paramètre `max-cache-ttl`, qui définit la durée de vie maximale des entrées conservées dans le cache. En fixant cette valeur à zéro, les entrées deviennent immédiatement invalides après leur insertion et ne peuvent donc pas être réutilisées pour satisfaire des requêtes ultérieures. Cette configuration permet ainsi d'émuler un fonctionnement du résolveur sans cache tout en conservant les autres mécanismes internes inchangés.

Entre chaque série de mesures, ce paramètre de configuration de Bind9 est modifié afin d'activer ou de désactiver le mécanisme de cache. Le serveur est ensuite redémarré afin de

prendre en compte la nouvelle configuration et de réinitialiser l'état du cache. En complément, la commande `rndc flush` est utilisée afin de vider explicitement le cache du résolveur.

Cette commande pourrait a priori constituer une solution suffisante pour réinitialiser l'état du cache. Toutefois, les expérimentations ont montré que son exécution n'entraîne pas de modification observable de la taille du cache. Pour cette raison, le redémarrage du service Bind9 s'avère nécessaire, à la fois pour garantir la prise en compte de la configuration modifiée et pour assurer une remise à zéro effective du cache entre les différentes séries de mesures.

3.3.2 Campagnes de mesures

Maintenant que l'environnement expérimental est configuré et que les outils de mesure sont en place, la phase suivante consiste à réaliser les campagnes de mesures. Pour ce faire, il faut définir les différentes configurations à tester, les protocoles à utiliser, les charges à appliquer, etc. Ensuite un faux orchestrateur pour lancer les différentes mesures de manière automatisée et synchronisée.

Il faut noter que étant donné que les campagnes de mesures sont généralement longues à réaliser, il est important de les planifier de manière efficace et de s'assurer que l'espace disque est suffisant pour stocker les données collectées. Il est également important de vérifier si la mesure est stable et fiable avant de lancer les campagnes de mesures à grande échelle, c'est ce qu'on appelle un sanity check, qui consiste à faire des mesures de test pour vérifier que tout fonctionne correctement et que les résultats obtenus sont cohérents avec les attentes. Cela est fait au préalable à chaque changement dans les scripts de mesure ou dans la configuration expérimentale. Une analyse rapide est effectuée.

Le script le plus important pour toutes les mesures et le script d'envoi des requêtes. Il prend en paramètres le protocole à utiliser et la charge à appliquer. Il existe deux versions de ce script, une pour les connexions persistantes et une pour les connexions non persistantes.

Mesures de consommation énergétique

En premier lieu, chaque mesure basée sur la consommation du résolveur DNS sera échantillonnée plusieurs fois (5 fois dans notre cas) afin d'obtenir des résultats plus fiables (réduire le bruit et les incertitudes) et de pouvoir calculer des intervalles de confiance. Pour les configurations sélectionnées, on retrouve les différentes combinaisons suivantes :

- **Commun pour les deux types de connexions :**
 - Protocoles : UDP, DoT, DoH
 - Utilisation du cache : avec cache, sans cache
- **Connexions persistantes :** une connexion TLS est établie au début de la mesure et est maintenue tout au long de la mesure.
 - QPS : allant de 30 QPS à 330 QPS avec 30 QPS d'intervalle entre chaque mesure
- **Connexions non persistantes :** une nouvelle connexion TLS est établie pour chaque requête DNS envoyée par le client.
 - QPS : allant de 50 QPS à 550 QPS avec 50 QPS d'intervalle entre chaque mesure

Pour l'orchestration des mesures, plusieurs scripts sont responsables de lancer les différentes mesures de manière automatisée et synchronisée. En partant du plus haut niveau au plus bas niveau, on trouve :

- `extensive_measurements.sh` : Script principal avec une boucle par paramètre de configuration (protocoles, utilisation du cache, QPS et échantillon), il lance avec ces paramètres `start.sh`.

- `start.sh` : script qui exécute une mesure unique pour une configuration donnée. Il vide le cache du DNS, change la configuration de Bind9 pour activer ou désactiver le cache, redémarre le service, crée les dossiers pour l'organisation des données récoltées, lance `start_dns_logs_monitoring.sh` sur le DNS puis lance `start_measurements.sh` sur le client.
- `start_dns_logs_monitoring.sh` : script qui exécute tous les scripts de mesures réseau et de logs DNS en parallèle.
- `start_measurements.sh` : script qui exécute tous les scripts de mesures systèmes, réseau et énergétiques en parallèle.
- `start_measurements.sh` : script qui exécute le script de mesure de consommation externe `yoctowatt_log_fast.py` en parallèle avec `orchestrate_measures.sh` côté DNS pour lancer tout les scripts de mesure de critères systèmes.

Des campagnes de mesures ont été réalisées pour mesurer la consommation énergétique du résolveur DNS avec les mêmes configurations mais sans effectuer de mesures au niveau du résolveur DNS pour voir l'influence de ces mesures sur la consommation énergétique du résolveur DNS. Également, la consommation énergétique du résolveur DNS a été mesurée au repos pour avoir une base de comparaison pour les mesures réalisées avec les différentes configurations.

Mesures de la complexité algorithmique

L'orchestration des mesures de la complexité algorithmique est bien plus simple car il ne faut lancer que le script d'envoi des requêtes et le script de mesure du nombre d'appels par fonction de Bind9. Cependant un seul échantillon est réalisé pour chaque configuration, car les mesures de la complexité algorithmique sont plus stables que les mesures de consommation énergétique, et il n'est pas nécessaire de faire plusieurs échantillons pour obtenir des résultats fiables.

La mise en place d'une infrastructure d'orchestration automatisée constitue une étape essentielle pour la réalisation de campagnes de mesures à grande échelle. Elle permet de garantir la reproductibilité des expérimentations, de limiter les erreurs humaines lors de l'exécution des protocoles et d'assurer la synchronisation des différentes sources de données collectées.

La répétition des mesures de consommation énergétique sur plusieurs échantillons permet par ailleurs de prendre en compte la variabilité inhérente aux systèmes informatiques et d'améliorer la robustesse statistique des résultats obtenus.

Enfin, la distinction entre les campagnes de mesures énergétiques et les campagnes d'analyse de la complexité algorithmique permet d'adapter les protocoles expérimentaux aux caractéristiques de chaque type de mesure. L'ensemble de cette méthodologie fournit ainsi un cadre expérimental rigoureux permettant d'obtenir des données fiables pour l'étude des performances énergétiques du résolveur DNS ainsi que pour l'élaboration des modèles présentés dans la suite de ce rapport.

3.3.3 Analyse des résultats

Les mesures obtenues ont été analysées principalement avec des Jupyter notebooks en utilisant pandas, numpy et matplotlib pour visualiser les résultats.

Ces analyses ont pour but de vérifier des hypothèses émises. La visualisation est essentielle pour valider ces hypothèses et comprendre les résultats obtenus, et repérer les erreurs

et anomalies dans les mesures.

Certains problèmes ont été rencontrés lors de l'analyse des résultats, notamment en raison du temps de traitement nécessaire, et la mémoire vive nécessaire pour analyser les grandes quantités de données collectées. Pour résoudre ce problème, des techniques d'échantillonnage ont été utilisées afin de réduire la taille des ensembles de données tout en conservant une représentativité suffisante pour les analyses. Cela réduit la précision des résultats, mais permet d'obtenir des analyses plus rapidement et de manière plus efficace. Un script a été développé pour automatiser ce processus d'échantillonnage et d'exportation des données dans un format plus facilement manipulable pour les analyses. Il rassemble toutes les métriques collectées lors des mesures, change la granularité à 1 seconde, remplit les données manquantes avec une méthode appropriée à la nature des données et les exporte dans un format CSV pour une utilisation ultérieure dans les notebooks d'analyse.

Grace à ce prétraitement des données automatisé, les résultats suivant ont pu être obtenus et analysés efficacement.

Le grand nombre de métriques collectées permet de faire des analyses approfondies des différents levier de consommation. Cependant, durant ce stage, ce grand panel de métriques n'a pas été exploité à son plein potentiel et par moment, il a été difficile de trouver quelles étaient les métriques les plus pertinentes à étudier pour ce sujet de recherche.

Cette section résume les résultats les plus intéressants obtenus et les analyses, en mentionnant les limitations de ces résultats s'il y en a, et les prochaines étapes pour les améliorer ou les approfondir.

Mesures de consommation énergétique du résolveur DNS

Les résultats les plus concrets de cette étude de consommation énergétique du résolveur DNS sont ceux montrant la consommation énergétique du résolveur pour les différents protocoles utilisés (UDP, DoT et DoH) avec et sans utilisation du cache pour différentes charges.

Pour les protocoles DoT et DoH, les mesures ont été réalisées avec des connexions persistantes et non persistantes. Pour les connexions persistantes, une connexion TLS est établie au début de la mesure et est maintenue tout au long de la mesure. Pour les connexions non persistantes, une nouvelle connexion TLS est établie pour chaque requête DNS envoyée par le client.

Il faut noter que pour DoH, les connexions sont systématiquement persistantes, car DoH utilise le protocole HTTP qui est conçu pour maintenir des connexions persistantes. Cependant, pour les besoins de cette étude, une configuration a été mise en place pour simuler des connexions non persistantes en fermant la connexion HTTP après chaque requête DNS. Cela permet de simuler le fait que chaque requête DoH correspond à un client différent.

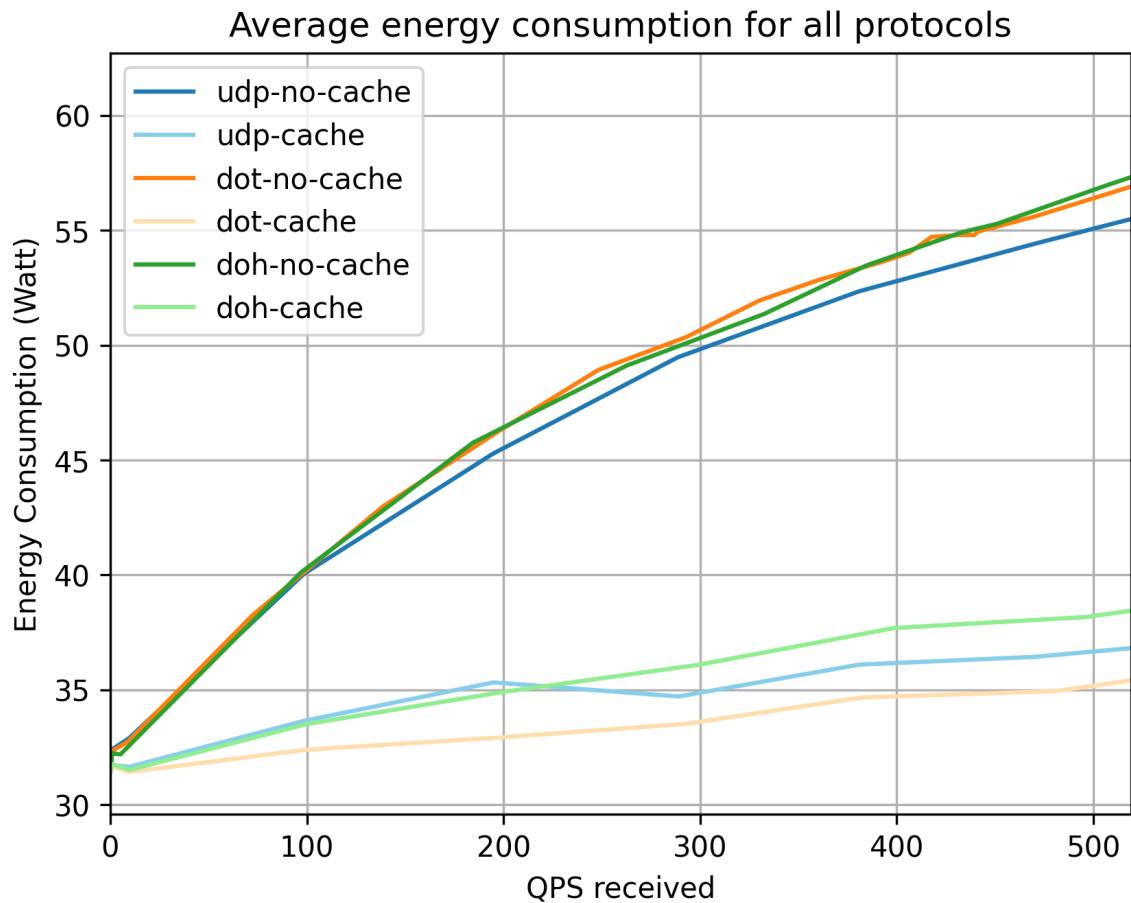


Figure 3.4 – Consommation énergétique moyenne pour UDP, DoT et DoH avec et sans utilisation du cache avec des connexions persistantes

Il existe une différence claire de consommation énergétique entre l'utilisation avec et sans cache. Les courbes pour l'utilisation sans cache ont une forme de décroissance exponentielle. En observant les courbes d'utilisation du cache, l'ordre est étrange : UDP n'utilise aucune connexion, il devrait donc être en dessous de toutes les autres courbes.

La différence entre les protocoles utilisant TLS (DoT et DoH) et le protocole utilisant UDP est due au coût du chiffrement principalement. Étant donné que pour ces mesures, les connexions sont persistantes, le coût de l'établissement de la connexion s'efface sur une longue période. Ces résultats sont donc cohérents avec ce qui est attendu.

Pour voir le coût de l'établissement des connexions, il faut faire des mesures avec des connexions non persistantes.

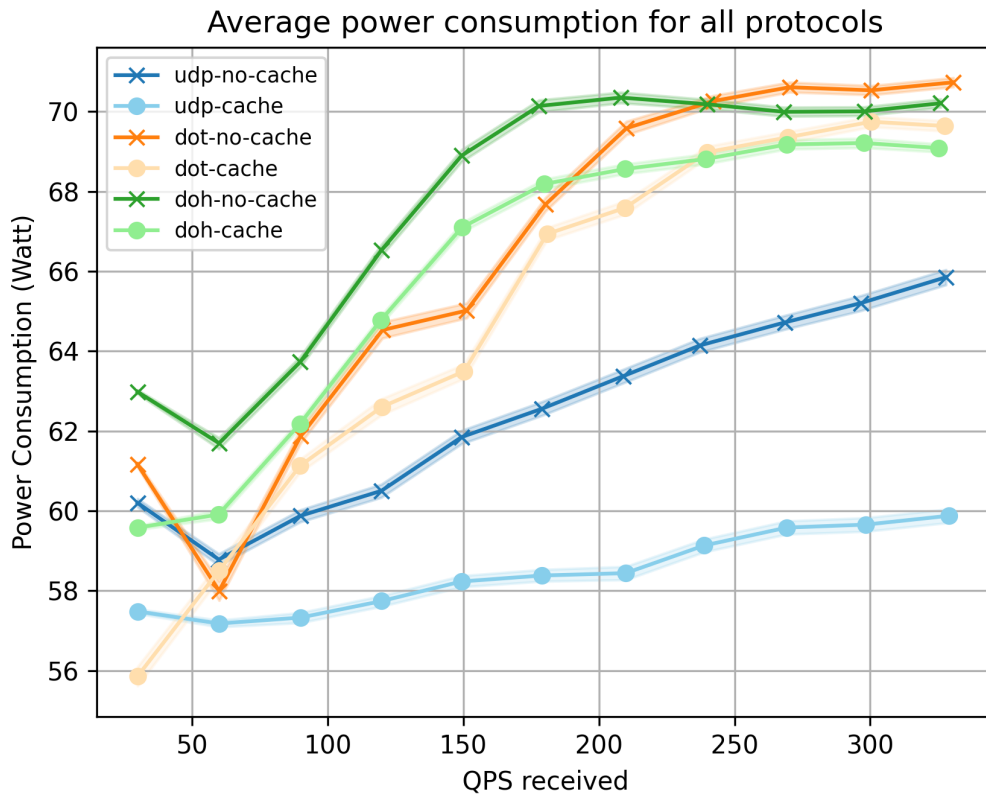


Figure 3.5 – Consommation énergétique moyenne pour UDP, DoT et DoH avec et sans utilisation du cache avec des connexions non persistantes

Avec des connexions non persistantes, la consommation énergétique est plus élevée pour les protocoles utilisant TLS (DoT et DoH) en raison du coût de l'établissement de la connexion pour chaque requête. Le protocole DoH consomme le plus ce qui est cohérent avec le coût supplémentaire du protocole HTTP en plus du chiffrement TLS.

Pour pouvoir interpréter ces résultats sans biais lié à la surcharge introduite par les scripts de mesure, il est nécessaire de faire des mesures de consommation énergétique sans la surcharge liée à l'enregistrement des métriques au niveau du DNS.

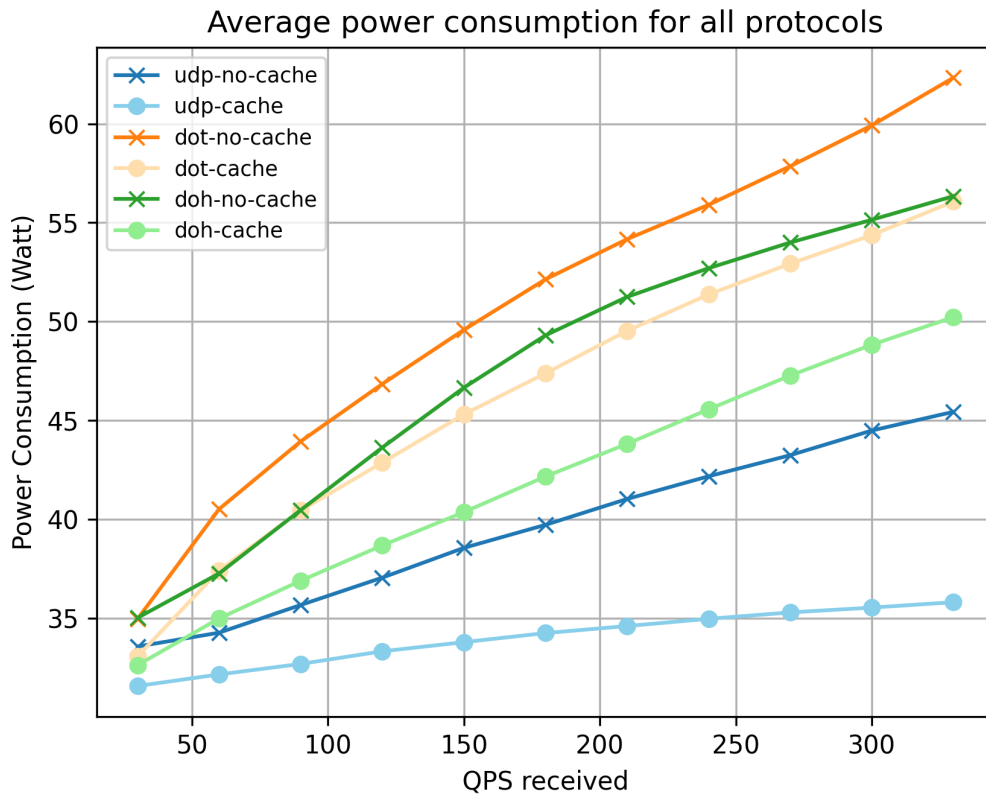


Figure 3.6 – Consommation énergétique moyenne pour UDP, DoT et DoH avec et sans utilisation du cache avec des connexions non persistantes, sans la surcharge lié à l'enregistrement des métriques au niveau du DNS

Étonnamment, les résultats sont différents, le protocole DoT consomme davantage que le protocole DoH, ce qui ne corrobore pas l'hypothèse selon laquelle DoH consomme plus que DoT en raison du coût supplémentaire du protocole HTTP. Comme le protocole mesuré était le même dans ces essais, cette différence provient probablement de la consommation des scripts de mesure au niveau du DNS, qui dépend donc du protocole utilisé.

Le QPS est limité pour DoT et DoH en raison de la surcharge introduite par l'établissement de connexions TLS pour chaque requête, ce qui rend les mesures à des QPS plus élevés difficiles à réaliser. De plus, il y a un nombre limité de socket disponibles pour les connexions TLS, ce qui peut également limiter la capacité à atteindre des QPS plus élevés. C'est encore plus flagrant avec les connexions non persistantes, où une nouvelle connexion TLS doit être établie pour chaque requête. DoH est le protocole avec le plus de surcharge, par conséquent, c'est le protocole avec lequel le QPS maximum est le plus bas.

Il serait intéressant d'avoir des mesures pour des QPS plus élevés et avec différentes configurations DNS (nombre maximal de clients du résolveur, nombre de threads de travail, etc.) et différentes implémentations DNS (PowerDNS, Unbound, Knot DNS, etc.).

Pour vérifier que le protocole utilisé pour les mesures représente bien ce que nous essayons de mesurer, il faut vérifier certaines métriques importantes. Le paramètre le plus crucial des configurations de mesure est l'utilisation ou non du cache. En effet, si le cache est utilisé, la consommation énergétique du résolveur est beaucoup plus faible, et les résultats sont très différents de ceux obtenus sans utilisation du cache. Par conséquent, il est essentiel de vérifier que le cache est bien utilisé ou non utilisé selon la configuration de me-

sure. Pour cela, il est nécessaire d'examiner les métriques du cache de Bind9. Les métriques collectées nous intéressant sont la place occupée par le cache, pour cette métrique il existe 2 façon de mesurer, il y a la mémoire utilisée par le tas du cache, c'est à dire les le contenu des données DNS et la mémoire utilisée par la structure arborescente du cache. Il y a également le nombre de requêtes qui touchent le cache, c'est à dire le nombre de requêtes pour lesquelles une réponse est trouvée dans le cache, et le nombre de requêtes qui ne touchent pas le cache. On peut déduire avec ces dernières le pourcentage de requêtes qui touchent le cache. Enfin une métrique importante qui révèle l'efficacité du protocole mis en place pour bien simuler la non utilisation du cache est l'expiration des entrées du cache du au TTL. En effet, comme mentionné précédemment, pour simuler la non utilisation du cache, le TTL maximum du cache est défini à 0, ce qui signifie que les entrées du cache expirent immédiatement après leur ajout. Par conséquent, si le cache est correctement configuré pour ne pas être utilisé, il devrait y avoir une grande quantité d'expiration d'entrées du cache due au TTL.

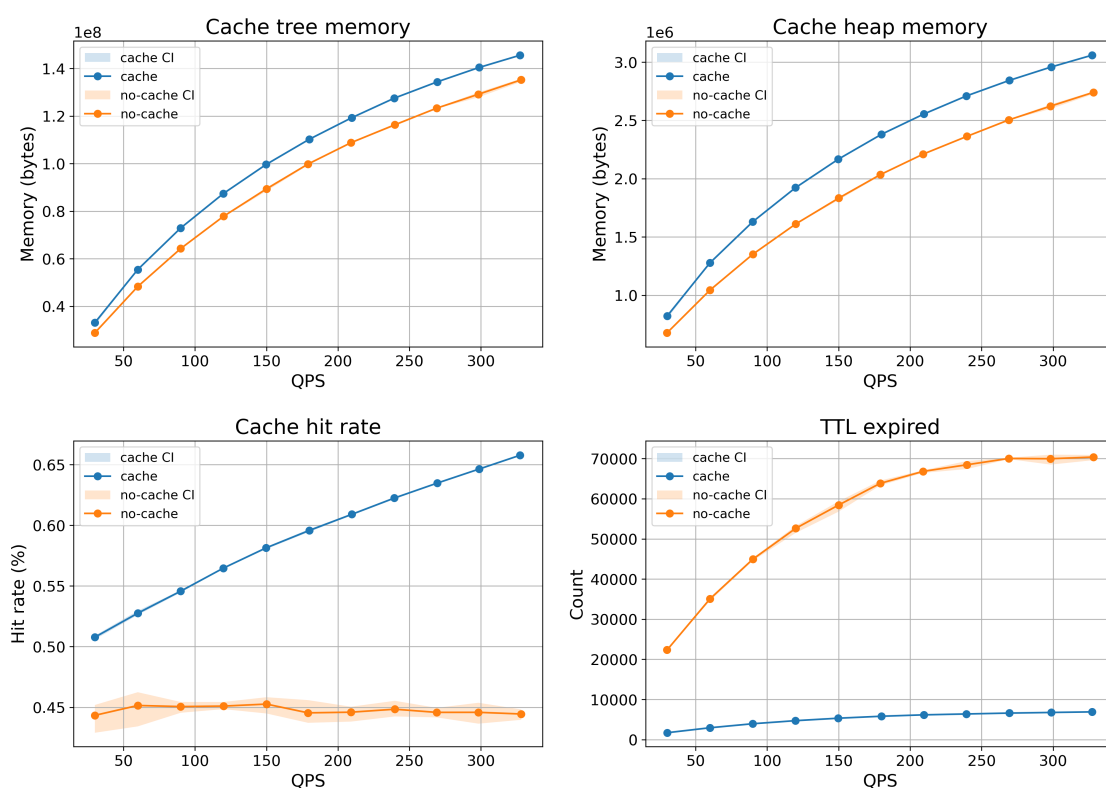


Figure 3.7 – Métriques du cache de Bind9 moyennées pour les différents protocoles avec et sans utilisation du cache

Concernant l'espace mémoire occupé par l'arborescence et le tas du cache, on peut voir une petite différence entre les mesures avec et sans utilisation du cache, mais cette différence n'est pas très grande, ce qui est surprenant. De plus, l'occupation de la mémoire augmente avec le QPS dans les deux cas, ce qui n'était pas attendu également. Cependant on voit, comme prévu au niveau du pourcentage de requêtes qui touchent le cache une différence claire entre l'utilisation du cache et la non utilisation du cache. De plus, pour les mesures n'utilisant pas de cache on voit une stagnation du pourcentage de requêtes qui touchent le cache vers 45% ce qui est plus que prévu. Les courbes d'expiration des entrées du cache dues au TTL montrent également une différence claire entre les mesures avec et sans utilisation du cache, avec une grande quantité d'expiration pour les mesures sans utili-

sation du cache et très peu pour l'utilisation du cache ce qui montre l'efficacité du paramètre `max-cache-ttl`.

Pour expliquer les incohérences observées, il faut noter que le protocole est faillible car la phase de remplissage du cache n'est pas homogène, elle devrait être faite avec un certain nombre de requêtes sur un ensemble de nom de domaine. Cependant il est fait pendant une certaine durée avec le QPS de la configuration, donc la durée de la mesure est étendue de 1 minute 30 (temps mesuré du remplissage du cache pour 100 000 noms de domaines 3.1) et la première minute 30 est ignorée pour les analyses. Cette minute 30 de remplissage a été mesurée avec un 10 000 QPS, le protocole n'est donc pas rigoureux et sera modifié pour les prochaines mesures par manque de temps, cela n'a pas été fait pour ces mesures. Cette faille dans le protocole explique la différence de d'occupation de la mémoire du cache avec l'utilisation du cache et l'augmentation du pourcentage de requêtes qui touchent le cache mais pas pourquoi autant d'espace est occupé par le cache même sans utilisation du cache et pourquoi ce pourcentage de requêtes qui touchent le cache est si haut sans utilisation du cache.

Concernant la mémoire utilisée par le cache lors des mesures faites sans cache, les entrées du cache sont ajoutées au cache mais expirent immédiatement après leur ajout, elles sont supprimées du cache par un autre processus qui vérifie régulièrement les quelles entrées ont expiré du au TTL. Ce processus de suppression des entrées expirées du cache n'est pas instantané et il est possible que ce soit pour cette raison que l'espace mémoire occupé par le cache soit du même ordre de grandeur avec et sans utilisation du cache. Pour expliquer le pourcentage de requêtes qui touchent le cache sans utilisation du cache, il faut noter que les requêtes qui touchent le cache sont celles pour lesquelles une réponse est trouvée dans le cache ou dans la zone (serveur root uniquement dans notre cas). Cette valeur est élevée mais n'augmente pas à la différence de l'utilisation du cache, ce qui montre un comportement différent. Cette valeur peut s'expliquer par le fait que l'adresse du serveur root est nécessaire pour toutes les résolutions.

Étude de la distribution des types de requêtes et des TTLs

Une manière de visualiser l'utilisation du résolveur consiste à examiner chaque paquet envoyé et reçu par le résolveur. Sachant que Bind9 utilise UDP pour communiquer avec les DNS autoritatifs, l'examen des paquets UDP est idéal. Ces paquets incluent de nombreux types de requêtes que nous verrons juste en dessous. Ces nombres de requêtes sont divisés par le QPS résolu par le résolveur.

Pour les mesures utilisant UDP, comme le résolveur utilise également UDP, pour éviter de mesurer les requêtes envoyées et reçues par le résolveur, nous déduisons 2 requêtes pour chaque résolution, ce qui correspond au modèle UDP.

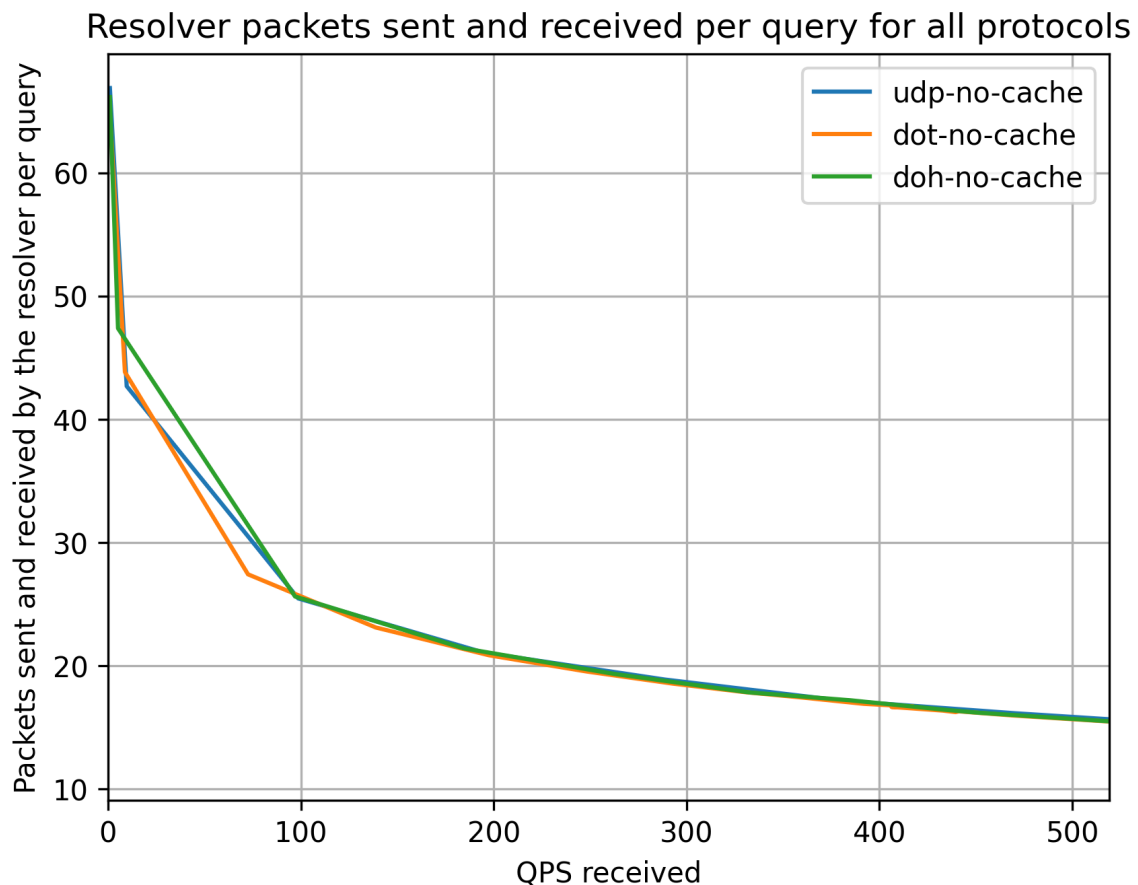


Figure 3.8 – Nombre de requêtes envoyées et reçues par le résolveur pour une résolution en fonction du QPS

Le fait que cela suive une décroissance exponentielle est étrange et doit être étudié. La manière d'étudier cela consiste à examiner la distribution des types de requêtes pour chaque valeur afin de voir quels types de requêtes sont moins présents lorsque le QPS augmente.

Toutes les requêtes récursives ne concernent pas des adresses IP : certaines sont des délégations vers un autre serveur autoritatif (NS), d'autres sont des alias vers un autre nom de domaine (CNAME). Il y a également des informations DNSSEC telles que RRSIG, DNSKEY, DS, etc.

Ainsi, pour voir la distribution de ces requêtes, voici un exemple pour une configuration.

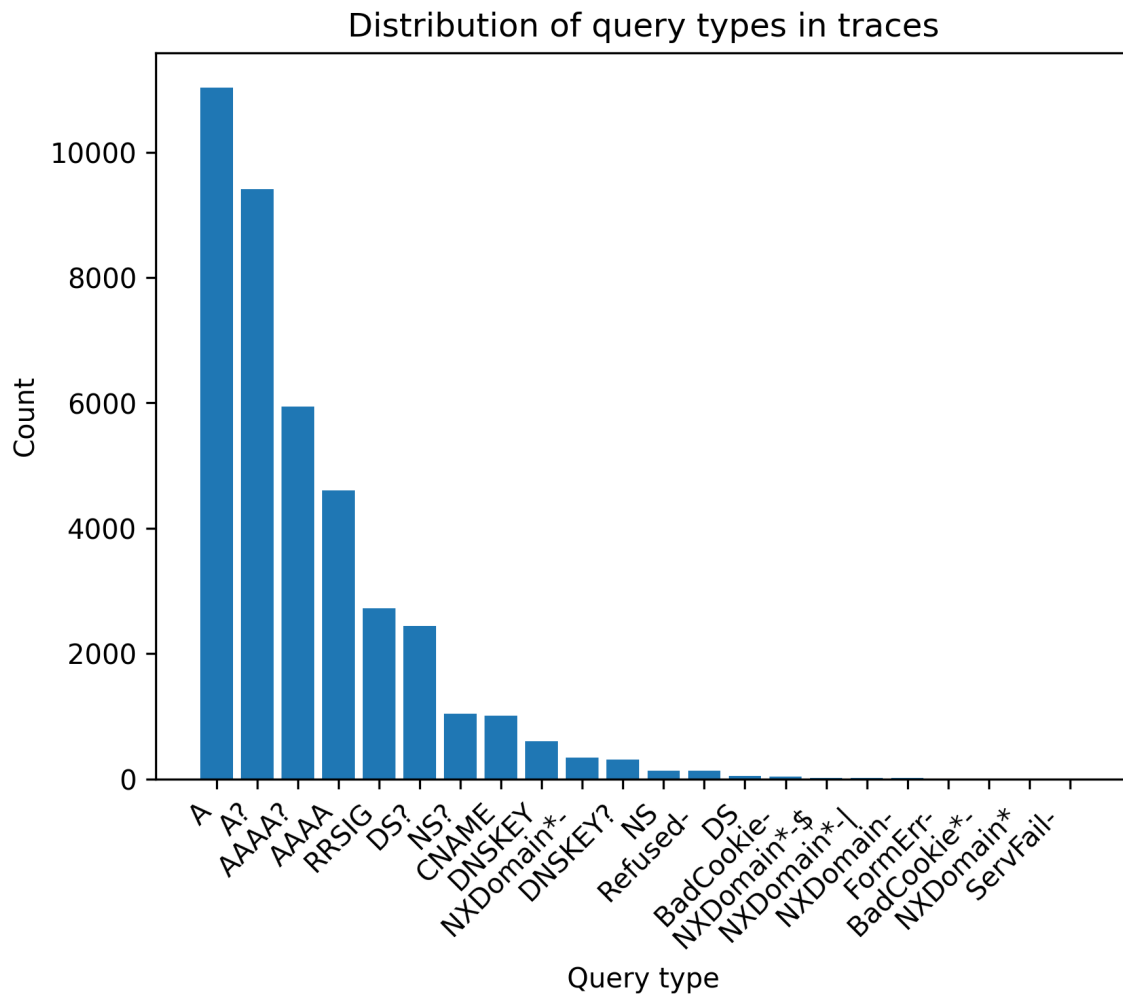


Figure 3.9 – Distribution des types de requêtes pour les résolutions récursives

La plupart sont de type A, mais il y a une part non négligeable de DNSSEC, NS et CNAME. Les types sont expliqués dans les annexe 3.4.

Le cache permet d'économiser beaucoup de consommation énergétique, mais les entrées du cache ont une durée limitée spécifiée par les serveurs DNS autoritatifs¹. Il s'agit du TTL, spécifié en secondes dans la zone pour chaque entrée sur laquelle le DNS a autorité. La distribution de ce TTL est intéressante à examiner.

Tout d'abord, nous pouvons examiner des noms de domaine aléatoires. Le graphique de gauche montre la CDF (Cumulative Distribution Function) de 100 000 noms de domaine aléatoires issus d'un ensemble d'1 million de noms de domaine pour avoir une idée de cette distribution. Le graphique de droite montre la distribution des TTL pour les TLD (Top Level Domains, comme .fr, .com), au total 1576 (le nombre exact n'est pas confirmé).

1. Cela est dû au fait que l'adresse IP peut changer au fil du temps; de plus, les DNS sont généralement utilisés comme une première couche de répartiteurs de charge

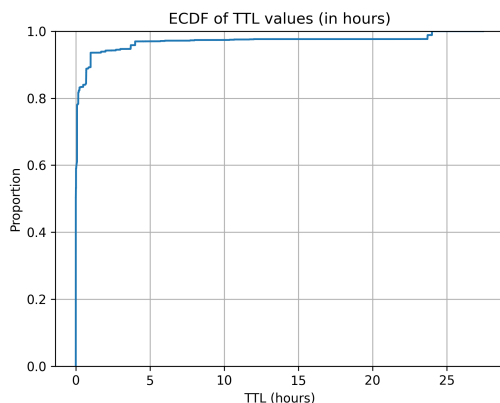


Figure 3.10 – Distribution des TTL pour des noms de domaine aléatoires

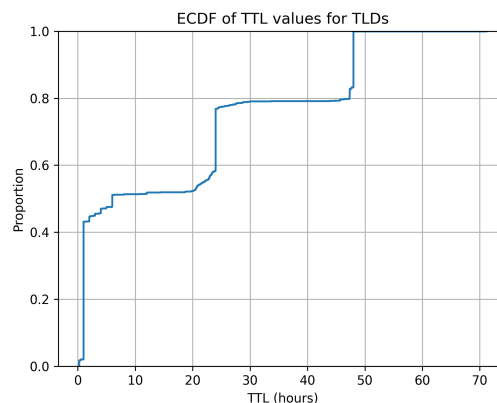


Figure 3.11 – Distribution des TTL pour les TLD

Pour les noms de domaine aléatoires, la distribution est très variable, mais il y a des pics à 1h, 4h et 24h, cependant la majorité des domaines ont un TTL de moins de 1 heure.

Pour les TLD, des pics peuvent être observés à 24h et 48h car ce sont des valeurs rondes. On remarque également que les TLD ont généralement des TTL plus élevés que les autres noms de domaine, ce qui est logique car ils auraient une charge plus importante étant donné qu'il n'y a pas beaucoup de TLD. Cela permet d'avoir une meilleure efficacité du cache pour les TLD, ce qui est important car ils sont souvent consultés.

Étude d'ablation

Étude de la complexité algorithmique de Bind9

Les mesures de consommation énergétique du résolveur DNS sont très intéressantes, mais elles ne permettent pas de comprendre les raisons de cette consommation énergétique. Pour mieux comprendre les raisons de cette consommation énergétique, et en s'abstrayant du matériel, une solution consiste à étudier la complexité algorithmique de Bind9. En effet, la consommation énergétique d'un programme est liée à sa complexité algorithmique, des paramètres de consommations peuvent être déduits à partir des performances énergétiques des composants de la machine (CPU, RAM, etc.).

L'étude de la complexité algorithmique de Bind9 s'est faite en deux parties, d'abord une étude de la complexité algorithmique statique de chaque fonction de Bind9 grâce à un outil développé en python par Rosario Patane, un de mes collègues et membre du projet DE-CODES. La seconde partie de l'étude de la complexité algorithmique de Bind9 s'est faite de manière dynamique, en mesurant le nombre d'appels par fonction de Bind9 pour différentes configurations de mesure. Cela a permis d'avoir une idée de la complexité algorithmique réelle de Bind9 pour différentes configurations, et ainsi d'avoir une meilleure compréhension des raisons de la consommation énergétique du résolveur DNS.

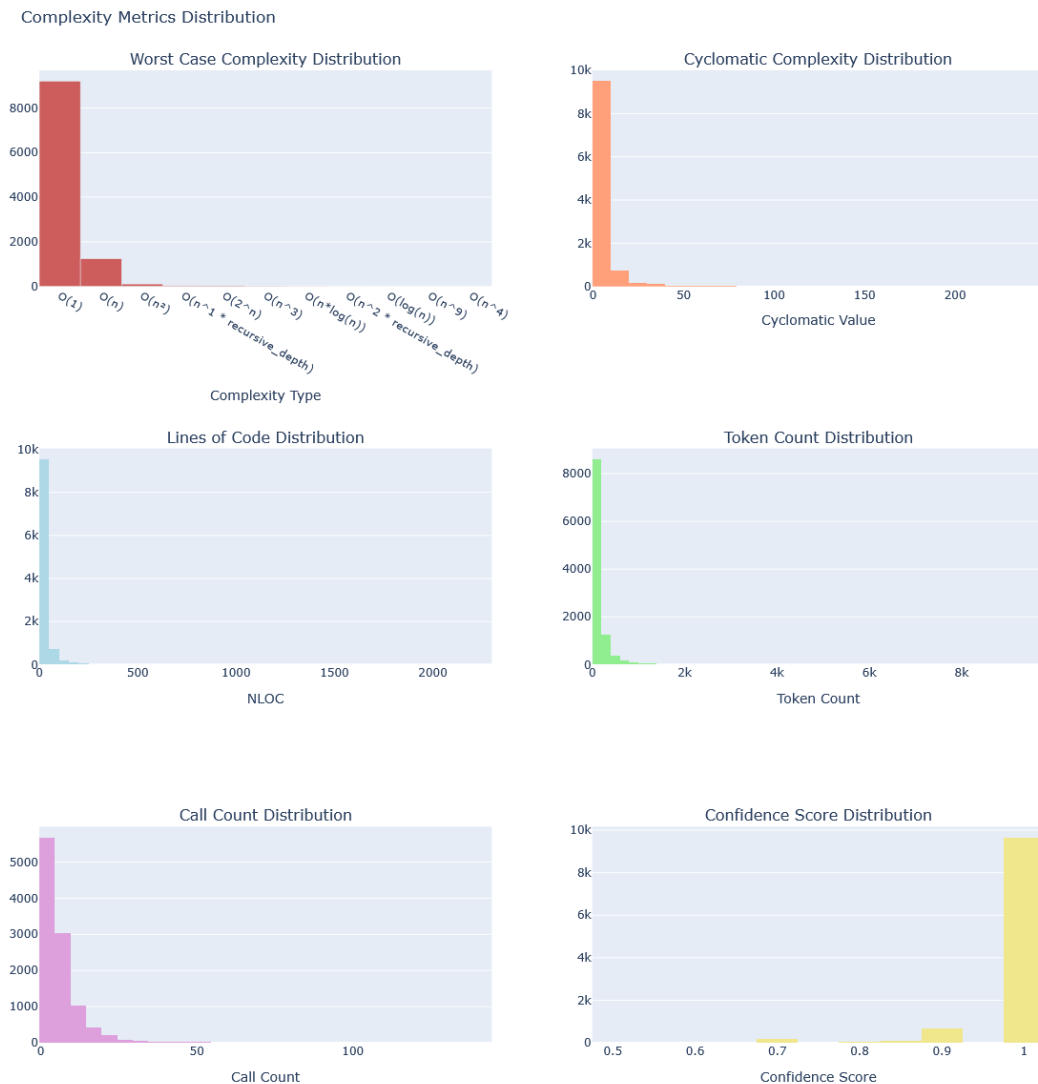


Figure 3.12 – Graphiques de l'analyse de la complexité algorithmique statique de Bind9

La figure 3.12 présente les résultats de l'analyse statique de la complexité algorithmique de Bind9 à travers six distributions complémentaires.

La première distribution représente les scénarios de pire cas (*Worst Case Scenario*) estimés pour les fonctions du code source. Elle met en évidence que la majorité des fonctions présentent une complexité faible, tandis qu'un nombre limité de fonctions se distingue par une complexité potentiellement élevée. Cette répartition est caractéristique des grands projets logiciels, dans lesquels une faible proportion de fonctions concentre une part importante de la complexité algorithmique.

La deuxième distribution concerne la complexité cyclomatique. Cette métrique mesure le nombre de chemins d'exécution indépendants à travers une fonction et constitue un indicateur de la complexité structurelle du code. Une valeur élevée traduit généralement la présence de nombreux branchements conditionnels, boucles ou mécanismes de gestion d'erreurs, rendant la fonction plus difficile à maintenir et à tester. Les résultats montrent que la majorité des fonctions de Bind9 possèdent une complexité cyclomatique relativement faible, tandis qu'un nombre restreint de fonctions présente des valeurs significativement plus élevées.

Les distributions du nombre de lignes de code et du nombre de jetons (*tokens*) permettent d'évaluer la taille des fonctions selon deux perspectives complémentaires. Alors que les

lignes de code donnent une estimation de la longueur des fonctions, le nombre de jetons fournit une mesure plus précise de leur richesse syntaxique. Dans les deux cas, les distributions mettent en évidence une forte concentration de petites fonctions et un nombre limité de fonctions particulièrement volumineuses.

La distribution du nombre d'appels entrants (*fan-in*) indique combien de fonctions différentes peuvent invoquer une fonction donnée. Cette métrique permet d'identifier les fonctions centrales dans l'architecture logicielle. Les résultats montrent que quelques fonctions jouent un rôle structurant au sein de Bind9 et sont réutilisées par de nombreuses autres fonctions.

Enfin, la distribution du score de confiance reflète le niveau de fiabilité attribué aux estimations de complexité produites par l'outil d'analyse statique. Les valeurs observées permettent d'évaluer la robustesse des résultats et d'identifier les estimations nécessitant une interprétation plus prudente.

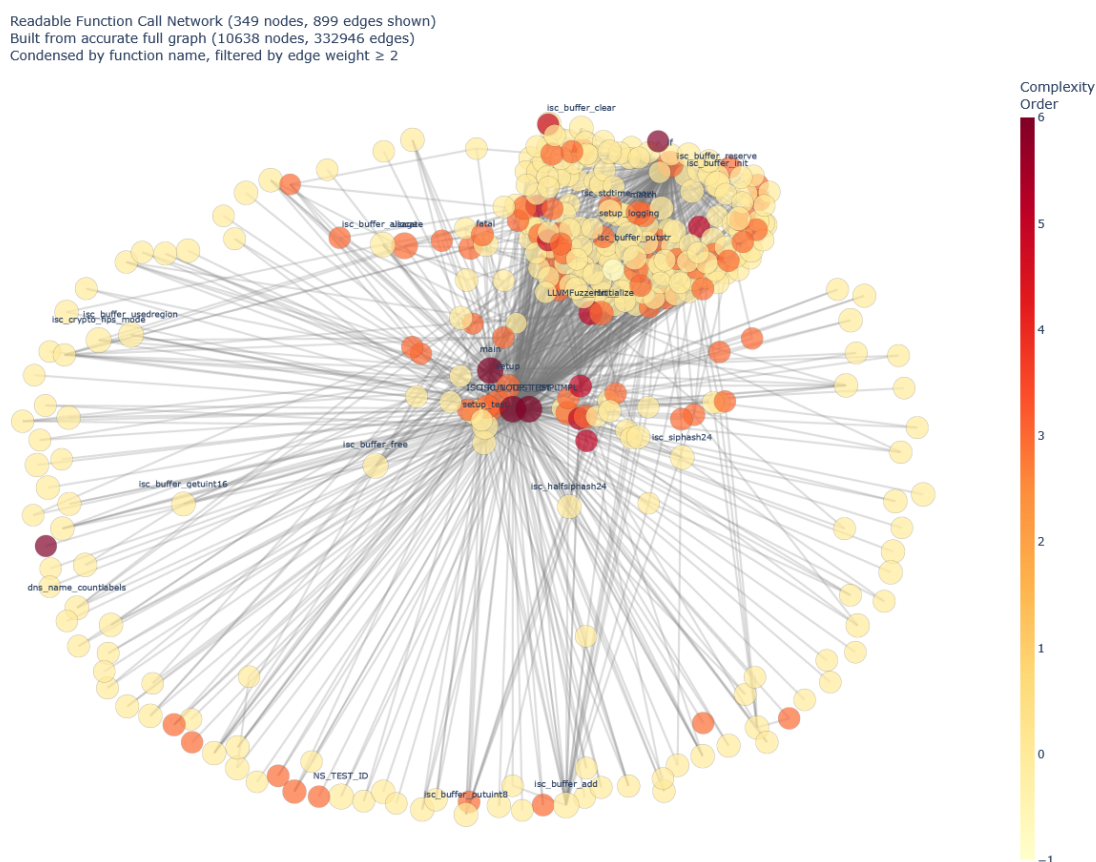


Figure 3.13 – Graph des relations d'appels entre les fonctions de Bind9 avec leur complexité algorithmique statique

La figure 3.13 représente le graphe des relations d'appels entre les fonctions de Bind9, construit à partir de l'analyse statique du code source et enrichi par les métriques de complexité algorithmique associées à chaque fonction. Chaque nœud correspond à une fonction et chaque arête représente une relation d'appel identifiée dans le code. Cette représentation permet de visualiser simultanément la structure des dépendances entre fonctions et la répartition de la complexité au sein du logiciel. Les fonctions fortement connectées apparaissent comme des éléments centraux de l'architecture, tandis que les fonctions présentant une complexité algorithmique élevée peuvent être identifiées comme des zones potentiellement plus difficiles à maintenir, tester ou analyser. Ce graphe fournit ainsi une vue d'ensemble de l'organisation statique de Bind9 et de la manière dont la complexité est distribuée dans son

code source.

La suite des graphiques se concentre sur l'analyse dynamique de la complexité algorithmique de Bind9.

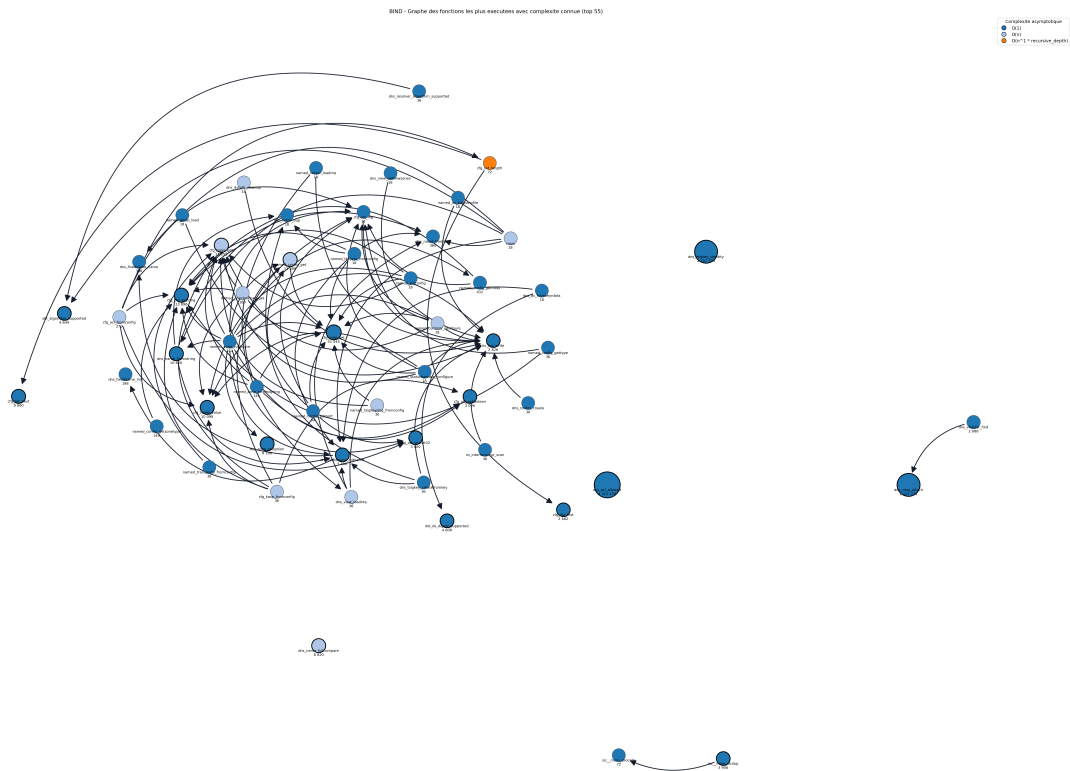


Figure 3.14 – Graph d'appels des 55 fonctions les plus appelées de Bind9 avec leur complexité algorithmique statique

La figure 3.14 se concentre sur les 55 fonctions recevant le plus grand nombre d'appels au sein de Bind9. Cette vue simplifiée facilite l'identification des fonctions critiques du système en supprimant les fonctions périphériques moins sollicitées. L'analyse met en évidence un nombre réduit de fonctions particulièrement centrales qui constituent les principaux points de passage lors de l'exécution du résolveur. Ces fonctions représentent des candidates privilégiées pour les analyses de performance et de consommation énergétique.

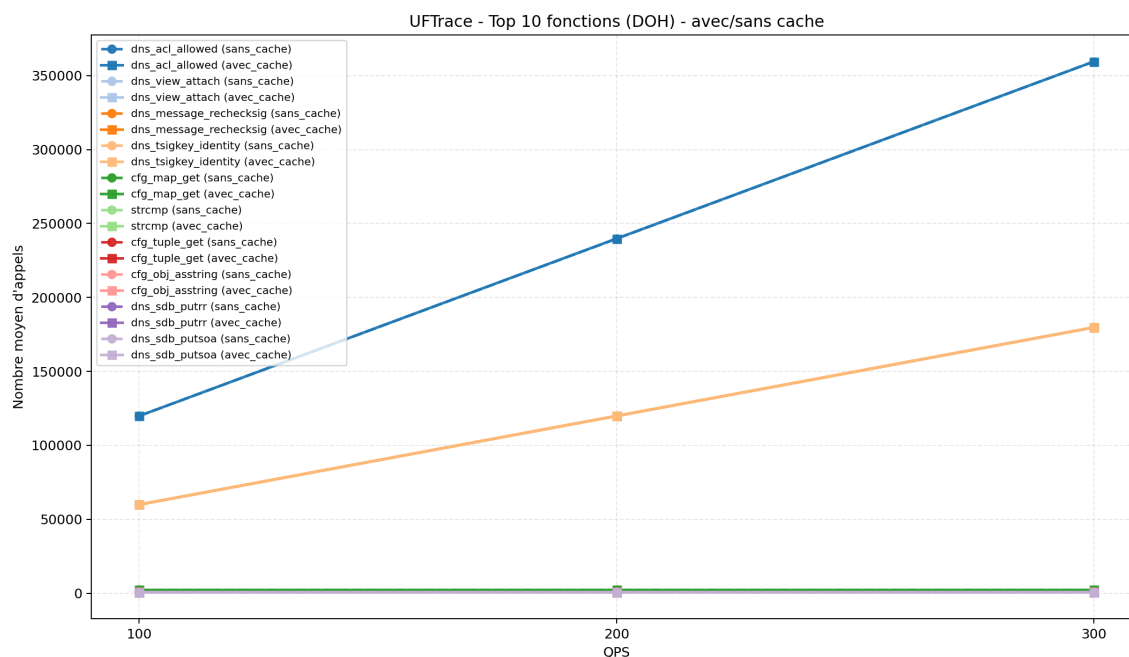


Figure 3.15 – Nombre d'appels des 10 fonctions les plus appelées de Bind9 en fonction du QPS pour DoH avec utilisation du cache

La figure 3.15 présente l'évolution du nombre d'appels des dix fonctions les plus sollicitées de Bind9 en fonction du débit de requêtes (QPS) dans le cas du protocole DoH avec utilisation du cache. Une augmentation globalement linéaire du nombre d'appels est attendue lorsque la charge augmente, traduisant la proportionnalité entre le trafic traité et l'activité interne du résolveur. Les différences observées entre les fonctions reflètent leur rôle respectif dans la pile de traitement DoH. Il n'y a aucune différence entre les mesures faites avec et sans cache, la différence doit se voir dans les temps d'exécution de ces fonctions cependant.

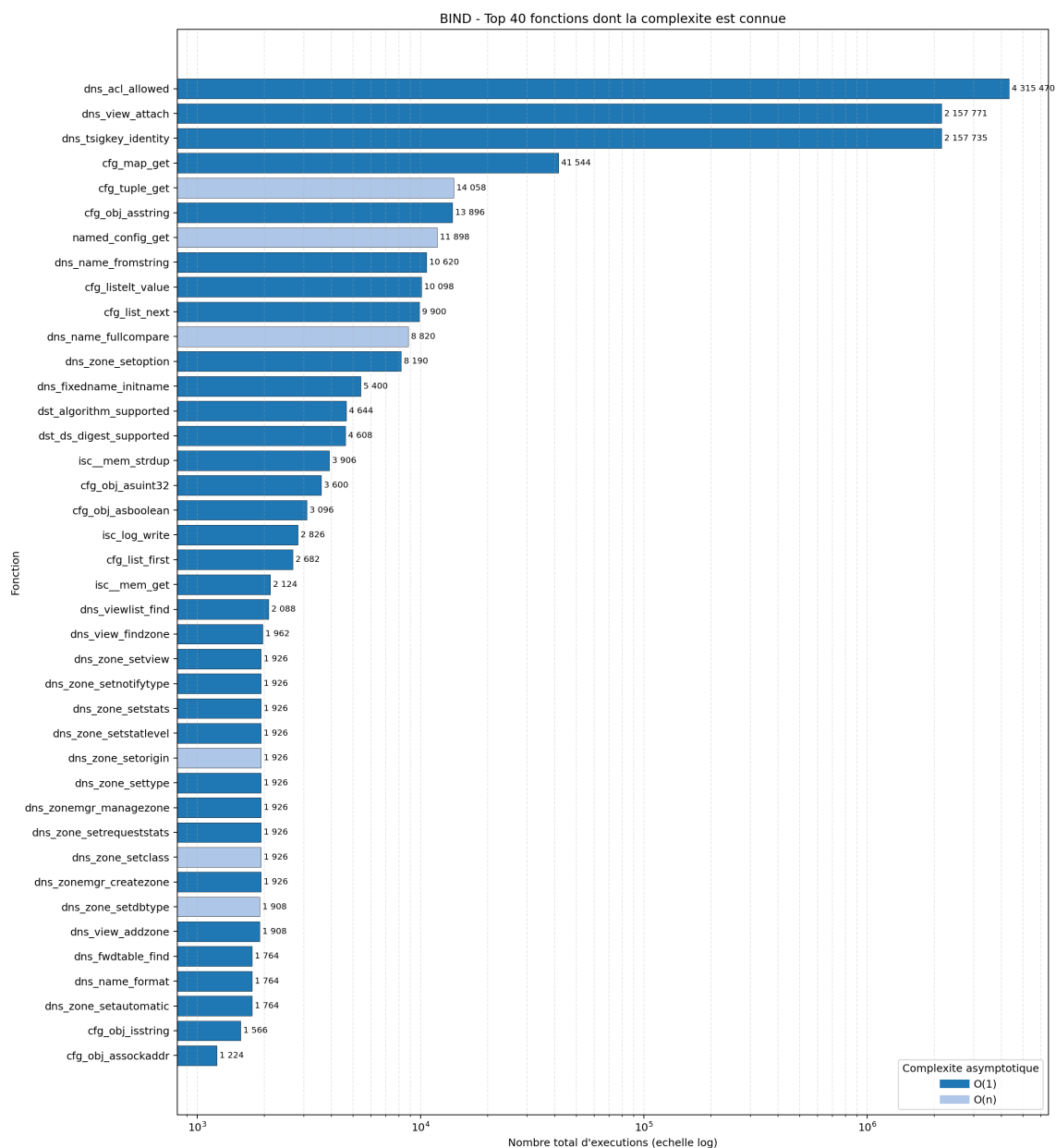


Figure 3.16 – Nombre d'appels des 40 fonctions les plus appelées de Bind9 avec leur complexité algorithmique statique

On remarque que 3 fonctions sont particulièrement sollicitées, à savoir `dns_acl_allowed`, `dns_view_attach` et `dns_tsigkey_identity`. Ces fonctions représentent à elle seule plus de 97% des appels effectués par Bind9. Elles semblent être appelées au moins une fois par requête. Voici une brève description de ces fonctions ainsi que de la fonction `cfg_tuple_get` qui est également très sollicitée, mais dans une moindre mesure.

- `dns_acl_allowed` (**$O(1)$**) : vérifie si une requête ou un client est autorisé selon les règles de contrôle d'accès (ACL) configurées dans Bind9.
- `dns_view_attach` (**$O(1)$**) : associe une nouvelle référence à une vue DNS (**view**) afin de permettre son partage sécurisé entre plusieurs composants du serveur.
- `dns_tsigkey_identity` (**$O(1)$**) : retourne l'identité (nom DNS) associée à une clé TSIG utilisée pour l'authentification des échanges DNS.
- `cfg_tuple_get` (**$O(n)$**) : recherche et récupère un élément dans une structure de configuration de type **tuple**. La recherche étant linéaire, son temps d'exécution est proportionnel au nombre d'éléments contenus dans la structure.

L'analyse statique et dynamique de la complexité de Bind9 met en évidence une répartition hétérogène des traitements au sein du résolveur. Si la majorité des fonctions présentent une complexité relativement faible et un nombre limité d'appels, un ensemble restreint de fonctions occupe une position centrale dans l'architecture du logiciel et concentre une part importante de l'activité observée lors du traitement des requêtes DNS.

L'étude des graphes d'appels permet d'identifier ces fonctions critiques ainsi que leurs relations avec les différents sous-systèmes du résolveur. Les mesures dynamiques montrent par ailleurs que certaines fonctions voient leur nombre d'appels évoluer proportionnellement à la charge appliquée, ce qui suggère un lien direct entre leur activité, les performances du résolveur et sa consommation de ressources.

Ces résultats fournissent une meilleure compréhension du fonctionnement interne de Bind9 et permettent d'identifier les composantes susceptibles d'avoir le plus fort impact sur les performances et la consommation énergétique. Ils constituent ainsi une base de travail pertinente pour la construction de modèles visant à représenter le comportement du résolveur.

Cependant, il manque une analyse du temps d'exécution de ces fonctions, qui pourrait donner des informations supplémentaires sur la consommation énergétique du résolveur. De plus, il est probable que nous observions des différences dans le temps d'exécution de ces fonctions selon la configuration de mesure (avec ou sans cache, protocole utilisé, etc.). Il est également important de noter qu'il manque probablement des fonctions critiques dans cette analyse, pour pouvoir mesurer les fonctions manquantes, il faudrait intégrer les bibliothèques utilisées par Bind9 dans l'analyse de la complexité algorithmique. Cela permettrait d'avoir une vue plus complète de la consommation énergétique du résolveur DNS.

Les différentes campagnes de mesures réalisées ont permis de caractériser le comportement du résolveur DNS sous plusieurs angles complémentaires : utilisation du cache, consommation des ressources système, consommation énergétique et complexité algorithmique. L'analyse des résultats met en évidence l'influence de paramètres tels que le protocole utilisé, la charge appliquée ou encore l'utilisation du cache sur les performances et la consommation du système.

Au-delà des observations quantitatives, ces expérimentations ont également permis d'identifier les principaux mécanismes impliqués dans le traitement des requêtes DNS et de mieux comprendre leur contribution relative à la charge du résolveur. Les métriques collectées constituent ainsi un ensemble de données riche permettant de relier l'activité interne de Bind9 aux ressources matérielles effectivement consommées.

Ces résultats constituent le socle expérimental nécessaire à la phase de modélisation. Les analyses précédentes ont permis d'identifier les variables pertinentes, d'estimer les paramètres caractéristiques du système et de valider les hypothèses formulées sur le fonctionnement du résolveur. La section suivante s'appuie sur ces observations afin de construire et d'évaluer différents modèles capables d'estimer la consommation énergétique d'un résolveur DNS à partir de ses caractéristiques de fonctionnement.

3.3.4 Modélisation

L'objectif principal de ce projet est de mieux comprendre et d'étudier la consommation énergétique des infrastructures DNS, en se concentrant plus particulièrement sur le résolveur Bind9. Les campagnes de mesures présentées dans les sections précédentes ont permis de constituer un ensemble de données comprenant à la fois des mesures de consommation énergétique, des métriques système et réseau ainsi que des informations relatives à la complexité algorithmique du logiciel.

Toutefois, les mesures seules ne permettent pas d'expliquer ou de prédire directement le

comportement énergétique du résolveur dans de nouvelles conditions d'utilisation. Une étape de modélisation est donc nécessaire afin de transformer ces observations expérimentales en outils capables d'estimer la consommation énergétique du système.

Deux approches complémentaires sont étudiées dans ce travail. La première repose sur des techniques d'apprentissage automatique permettant d'établir une relation entre les métriques observées et la consommation énergétique mesurée. La seconde s'appuie sur un modèle analytique fondé sur les réseaux de files d'attente, dont l'objectif est de représenter les différentes étapes du traitement d'une requête DNS et leur contribution à la consommation globale.

Ces approches répondent à des problématiques différentes mais complémentaires. Les modèles statistiques visent principalement à maximiser la précision des prédictions, tandis que les modèles analytiques cherchent à fournir une compréhension plus fine des mécanismes internes responsables de la consommation énergétique. Enfin, un troisième axe de recherche, basé sur l'analyse de la complexité algorithmique du résolveur, est actuellement à l'étude mais n'a pas encore abouti à un modèle exploitable.

Prédiction de la consommation énergétique du DNS

Les campagnes de mesures réalisées ont permis de collecter un grand nombre de métriques caractérisant l'activité du résolveur DNS, notamment l'utilisation du processeur, de la mémoire, du réseau, des entrées-sorties ainsi que différents indicateurs liés au fonctionnement du cache. Ces informations constituent une base de données riche permettant d'envisager une approche par apprentissage automatique.

L'objectif de cette approche est de construire un modèle capable de prédire la consommation énergétique du résolveur à partir des métriques observées. Un tel modèle présente plusieurs intérêts. D'une part, il permet d'estimer rapidement la consommation énergétique sans avoir recours à un wattmètre externe. D'autre part, il permet d'identifier les métriques les plus influentes dans la consommation observée et d'évaluer leur contribution relative.

Cette approche repose toutefois sur l'hypothèse que les métriques nécessaires à la prédiction sont disponibles au moment de l'estimation. Or, dans certains contextes de déploiement ou d'exploitation, l'accès à ces informations peut être limité. Il est donc important d'évaluer à la fois la précision du modèle obtenu et sa dépendance aux données d'entrée utilisées.

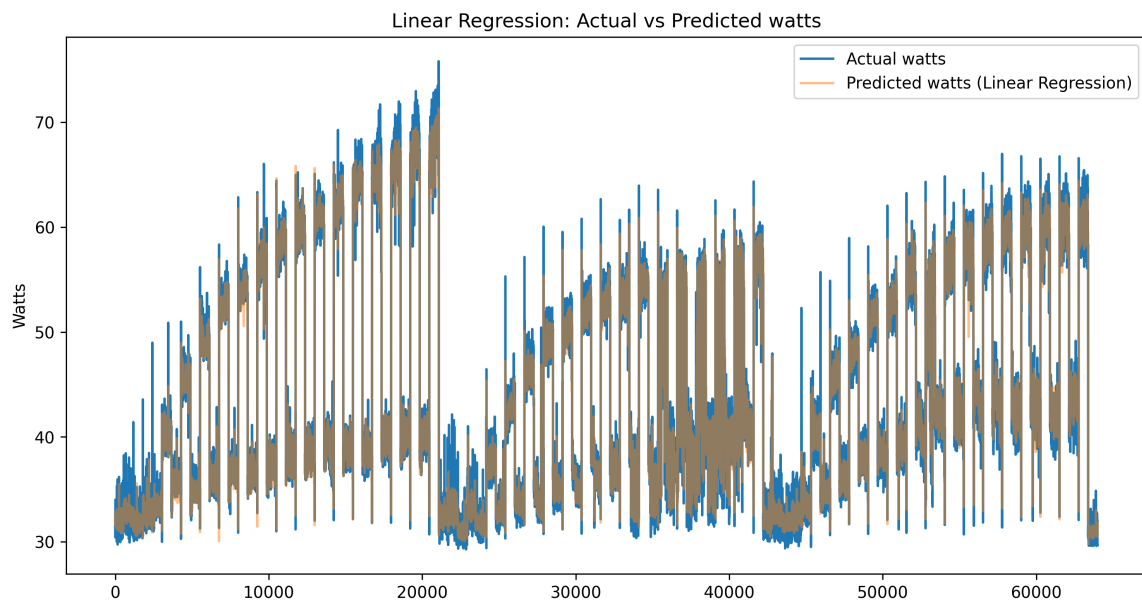


Figure 3.17 – Estimation par régression linéaire et consommation énergétique réelle

Modèle de réseau de files d'attente

Une approche permettant d'estimer la consommation énergétique d'un résolveur DNS consiste à décomposer le processus de résolution en plusieurs étapes élémentaires et à évaluer la contribution énergétique de chacune d'entre elles. Contrairement à une approche purement empirique, dépendante du matériel ou de l'implémentation utilisée, cette méthode vise à construire un modèle plus général permettant d'estimer la consommation d'un résolveur dans différentes conditions d'utilisation.

Les réseaux de files d'attente constituent un cadre particulièrement adapté à cet objectif. Ils permettent de représenter les différentes ressources mobilisées lors du traitement d'une requête ainsi que les délais associés aux temps d'attente et aux temps de service. Chaque étape du processus de résolution peut ainsi être modélisée indépendamment avant d'être intégrée dans un modèle global.

À partir de notre compréhension du fonctionnement interne de Bind9, un modèle de réseau de files d'attente a été élaboré. Celui-ci représente les principales étapes impliquées dans le traitement d'une requête DNS, depuis l'établissement éventuel de la connexion jusqu'à la génération de la réponse. Les différentes composantes du modèle sont les suivantes :

- Établissement de la connexion TCP (pour DoT et DoH) ;
- Handshake TCP ;
- Handshake TLS et échange des certificats ;
- Résolution récursive composée de :
 - l'exécution par les threads de travail du résolveur ;
 - les échanges DNS avec les serveurs autoritatifs ;
- Recherche et insertion dans le cache DNS.

Chacune de ces composantes est représentée par une file d'attente de type M/M/C. Les paramètres associés à chaque file devront être estimés expérimentalement afin de caractériser leur comportement. Les principales caractéristiques recherchées sont :

- ρ : taux d'utilisation ou charge de la ressource ;
- S : temps moyen de service.

L'estimation de ces paramètres repose sur une étude d'ablation. Cette méthodologie consiste à partir d'une configuration minimale mobilisant le moins de mécanismes possible,

puis à ajouter progressivement les différentes étapes du processus de résolution. L'objectif est d'isoler la contribution de chaque composante à la latence observée ainsi qu'à la consommation des ressources matérielles, notamment le processeur et la mémoire.

Les protocoles de mesure envisagés pour caractériser les différentes composantes sont les suivants :

- **Threads de travail** : mesure réalisée à l'aide de requêtes UDP dont les enregistrements sont déjà présents dans le cache afin d'éliminer le coût de la résolution récursive ;
- **Recherche dans le cache** : mesure réalisée avec des requêtes UDP en faisant varier le taux de remplissage du cache afin d'évaluer l'impact de sa taille sur les temps d'accès ;
- **Résolution récursive et requêtes vers les serveurs autoritatifs** : mesure réalisée avec des requêtes UDP portant sur des noms de domaine absents du cache ;
- **Établissement de la connexion TCP** : mesure réalisée à l'aide de requêtes TCP sans couche TLS afin d'isoler le coût de la connexion ;
- **Handshake TLS et échange des certificats** : mesure réalisée en comparant les performances obtenues avec TCP seul et avec DoT ou DoH, permettant d'isoler le surcoût induit par la couche TLS.

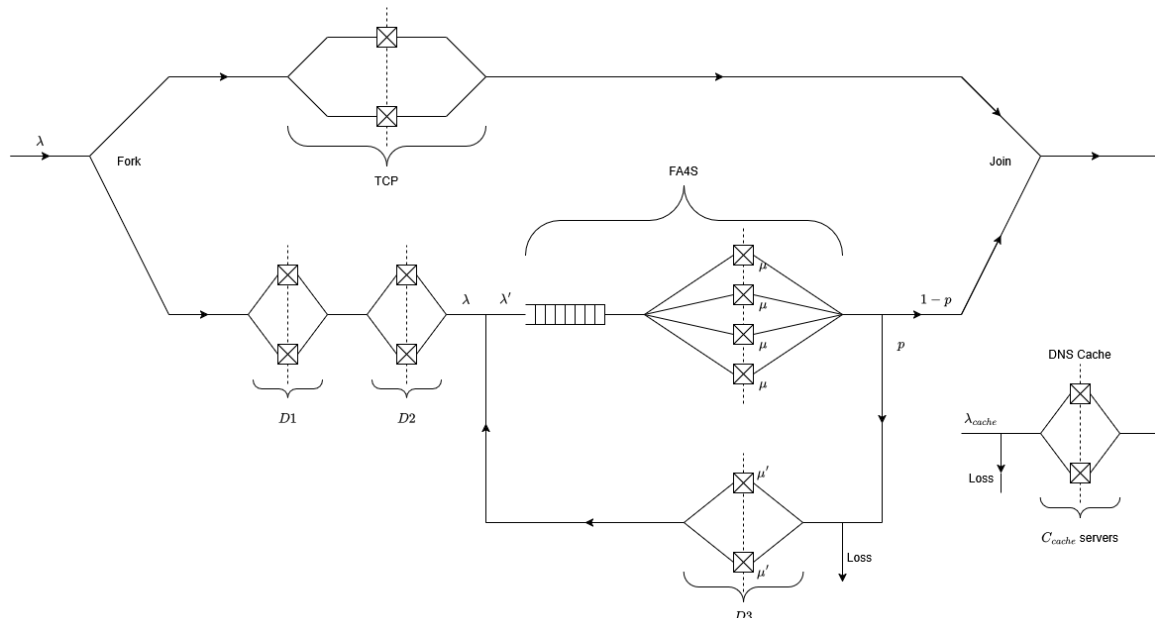


Figure 3.18 – Modèle de réseau de files d'attente représentant les principales étapes du traitement d'une requête DNS.

Ce modèle repose sur notre compréhension actuelle du fonctionnement interne de Bind9. Bien qu'il ne prétende pas représenter l'intégralité des mécanismes mis en œuvre par le résolveur, il fournit un cadre méthodologique permettant d'analyser et d'estimer sa consommation énergétique. Les travaux futurs consisteront à mesurer les paramètres associés à chaque composante du modèle, puis à valider expérimentalement leur capacité à reproduire le comportement observé du résolveur dans différentes conditions de charge et de configuration.

Analyse de complexité algorithmique

Petite introduction / Pourquoi voulons-nous utiliser la complexité algorithmique pour ce problème ?

Comment cela est-il fait ?

Petite conclusion / Quel est le principal résultat de cela ? John Doe. This is a book. Sous la direction d'Editor. Publisher, 2023 **test**

Conclusion

Bibliographie

Doe, John. This is a book. Sous la direction d'Editor. Publisher, 2023.

— This is a book. Sous la direction d'Editor. Publisher, 2023.

— This is a book. Sous la direction d'Editor. Publisher, 2023.

Annexes

Détail des métriques collectées

Nouvelles métriques

Le cache sera surveillé à l'aide de 6 variables disponibles via `rndc`, un module utilisé pour visualiser les métriques de Bind9.

Les nouvelles métriques sont :

Accès au cache

Le nombre total de requêtes DNS **servies directement depuis le cache**.

Cela inclut les réponses positives (par exemple, les enregistrements A, AAAA, MX) qui ont été précédemment mises en cache.

Unité : Compteur (entier non signé).

Échecs de cache

Le nombre total de requêtes DNS **non servies depuis le cache**, nécessitant une résolution externe (par exemple, en interrogeant un serveur DNS autoritatif ou récursif).

Unité : Compteur (entier non signé).

Accès au cache (à partir des requêtes)

Le nombre de **réponses en cache** pour les requêtes entrantes.

Contrairement aux accès au cache, cette métrique se concentre sur les **requêtes initiales** servies depuis le cache.

Unité : Compteur (entier non signé).

Échecs de cache (à partir des requêtes)

Le nombre de requêtes entrantes **non résolues par le cache**, nécessitant une résolution externe.

Complémentaire aux accès au cache (à partir des requêtes) pour évaluer l'efficacité du cache sur les requêtes entrantes.

Unité : Compteur (entier non signé).

Mémoire de l'arbre de cache

Mémoire **utilisée par la structure arborescente du cache** (en octets).

Représente la mémoire allouée pour organiser les entrées du cache dans une structure arborescente (optimisée pour des recherches rapides).

Unité : Octets.

Mémoire du tas de cache

Mémoire **utilisée par le tas du cache** (en octets).

Représente la mémoire allouée dynamiquement pour les entrées du cache non organisées dans l'arbre.

Unité : Octets.

Entrées de cache supprimées en raison de l'épuisement de la mémoire

Le nombre d'**entrées de cache supprimées** en raison de l'**épuisement de la mémoire**.

Indique que le cache a atteint sa limite de mémoire allouée (`max-cache-size`) et a dû supprimer des entrées pour libérer de l'espace.

Unité : Compteur (entier non signé).

Note : Une valeur > 0 suggère que `max-cache-size` devrait être augmentée.

Entrées de cache supprimées en raison de l'expiration du TTL

Le nombre d'**entrées de cache supprimées** car leur **TTL (*Time To Live*) a expiré**.

Représente le nombre d'enregistrements DNS automatiquement supprimés du cache après avoir atteint leur durée de vie maximale.

Unité : Compteur (entier non signé).

`Time_Of_Day_Seconds` : Wall-clock timestamp of the sample (seconds since midnight).
`Avg_MHz` : Average CPU frequency including idle time. `Busy%` : Percentage of time the CPU cores were active (not idle). `Bzy_MHz` : Average CPU frequency while cores were busy. `C6%` : Percentage of time cores spent in deep sleep (C6). `CoreTmp` : Average CPU core temperature (°C). `PkgTmp` : Temperature of the full CPU package (°C). `PkgWatt` : Total CPU package power consumption (watts). `CorWatt` : Power consumed by CPU cores only (watts). `LLCkRPS` : Total last-level cache (LLC) accesses per second, in thousands (loads + stores). `LLC%hit` : Percentage of LLC accesses that hit the cache instead of going to DRAM.

3.4 Types de requêtes DNS

Sujet de stage et description du travail réalisé

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Télécom SudParis

19 Place Marguerite Perey
91120 Palaiseau

www.telecom-sudparis.eu